LIGHTWEIGHT REAL-TIME DETECTION OF SPEED BUMPS AND POTHOLES

A Mini Project Report

U23ADS705 – MINI PROJECT

Submitted by

BENADICT INFANT A (61782323110022)
ELANGOVAN S (61782323110035)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SONA COLLEGE OF TECHNOLOGY (AUTONOMOUS), SALEM – 636005

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

SONA COLLEGE OF TECHNOLOGY (AUTONOMOUS)
ANNA UNIVERSITY: CHENNAI – 600 025
BONAFIDE CERTIFICATE

Certified that this project titled **LIGHTWEIGHT REAL-TIME DETECTION OF SPEED BUMPS AND POTHOLES** is the bonafide work of **BENADICT INFANT A (61782323110022), ELANGOVAN S (61782323110035)** who carried out the **U23ADS705 – MINI PROJECT** under my supervision.

Signature
Dr. P. SHANMUGARAJA
Supervisor
Assistant Professor,
Department of
Information Technology,
Sona College of Technology,
Salem – 636005.

Signature
Dr. J. AKILANDESWARI
Head of the Department
Professor,
Department of
Information Technology,
Sona College of Technology,
Salem – 636005.

Submitted for the Project Viva-Voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Sona College of Technology

Department of Information Technology

U23ADS705 – MINI PROJECT – DECLARATION FORM

Declaration by the Student(s)

We hereby declare that the project titled “Lightweight Real-Time Detection of Speed Bumps and Potholes” is an original work carried out by us under the guidance of Dr. P. Shanmugaraja. This project has not been submitted elsewhere for the award of any degree.

We also affirm that all the data and information used in this project are genuine and properly acknowledged. This project work is carried out with reference to the United Nations Sustainable Development Goals **Goal 11 – Sustainable Cities and Communities**.

S.No	Register Number	Name	Signature
1.	61782323110022	BENADICT INFANT A	
2.	61782323110035	ELANGOVAN S	

Date: _____

Place: Sona College of Technology

ACKNOWLEDGMENT

We would like to place on record our sincere thanks to the Management of Sona College of Technology, Salem, for providing the facilities and the academic environment in which this project was carried out.

We express our sincere thanks to our Chairman, **Sri. C. Valliappa**, and our Principal, **Dr. S. R. R. Senthilkumar**, for providing the facilities needed to complete this project.

We thank **Dr. J. Akilandeswari**, Head of the Department of Information Technology, for supporting this work and for making the department's resources available to us whenever we needed them.

We express our deep gratitude to our project guide, **Dr. P. Shanmugaraja**, Assistant Professor, Department of Information Technology, for his guidance at every stage of this project. His questions about what each part of the model actually contributes pushed us to look past the headline numbers, and that shaped the final form of this work.

We also thank the project coordinators and all the faculty members of the department for their review comments, which helped us sharpen both the experiments and this report.

We also acknowledge the open-source Ultralytics framework and the contributors of the public Humps/Bumps and Potholes dataset, on which the models in this project were built and evaluated.

Finally, we thank our parents and friends for their patience and support throughout the project.

BENADICT INFANT A

ELANGOVAN S

ABSTRACT

Road surface anomalies such as speed bumps and potholes pose major hazards to vehicular safety, ride comfort, and suspension longevity, particularly on non-standardised road networks where warning markings are frequently degraded or absent. Timely detection of these anomalies via forward-facing camera perception enables early driver warning and active suspension preview control, allowing electromagnetic or hydraulic dampers to adjust before tire-hazard impact. This project presents GV-YOLO26, an ultra-lightweight, real-time end-to-end hazard detector engineered specifically for small and distant road anomalies. Built on the modern anchor-free YOLO26n base, GV-YOLO26 eliminates conventional dense convolutional stages in favor of Ghost convolutions and VoV-style Group-Shuffle CSP (VoVGSCSPC) modules in the backbone and neck, substantially curtailing parameter redundancy and memory access overhead. To overcome the failure mode of deep detectors on tiny, distant road hazards, the architecture incorporates a dedicated high-resolution P2 feature pyramid branch coupled with a Lightweight Grouped Detection Head (LGDH) that mitigates multi-scale channel expansion through grouped parameter sharing. The network is trained using a composite bounding-box objective combining Task-Aligned CIoU with Normalized Gaussian Wasserstein Distance (NWD) loss to deliver continuous, non-vanishing position gradients on miniature hazard boundaries, supplemented with synthetic adverse-weather augmentations. Detections across sequential video streams are linked and verified through ByteTrack multi-object association, incorporating adaptive local contrast enhancement for low-light night driving and multi-frame temporal confirmation to reject transient false alarms. The proposed framework establishes that specialized small-object architectural re-engineering and native one-to-one NMS-free inference achieve exceptional edge deployment efficiency without compromising small-hazard sensitivity.

TABLE OF CONTENTS

TITLE	PAGE NO.
ACKNOWLEDGMENT	iv
ABSTRACT	v
LIST OF FIGURES	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS	x
1. INTRODUCTION	**1**
1.1 Background of the Project	1
1.2 Problem Statement	2
1.3 Motivation	2
1.4 Objectives	3
1.5 Scope of the Project	4
2. LITERATURE REVIEW	**5**
2.1 Introduction	5
2.2 Review of Related Studies	6
2.3 Techniques and Technologies Used in Existing Systems	16
2.4 Research Gap	20
2.5 Summary	21
3. CHAPTER 1 - NEED FOR PROPOSED TECHNIQUE	22
3.1 Existing Techniques	22
3.2 Limitations of Existing Techniques	23
3.3 Proposed Technique	24
3.4 Objectives of the Proposed Technique	24
3.5 Advantages of the Proposed Technique	25
4. CHAPTER 2 - DESIGN OF PROPOSED SYSTEM AND ITS IMPLEMENTATION	26
4.1 Introduction	26
4.2 System Requirements	26
4.2.1 Hardware Requirements	26
4.2.2 Software Requirements	27
4.3 System Architecture	27
4.4 System Components/Modules	33
4.4.1 Data Acquisition and Augmentation Module	33
4.4.2 Lightweight Backbone and Neck Module	33
4.4.3 Small-Object Detection Head Module	33
4.4.4 Training and Distillation Module	33
4.4.5 Deployment and Inference Module	34
4.4.6 Baseline Benchmarking Module	34
4.5 Proposed Methodology	34
4.6 System Workflow	36
4.7 Algorithms/Techniques Used	38
4.8 Dataset/Data Collection	43
4.9 Data Preprocessing	45
4.10 Model/System Development	45

4.11 System Design Diagrams 47
4.12 Implementation 49
4.13 Coding/Program Implementation 50
4.14 User Interface / Application Screens 56
4.15 Chapter Summary 57
5. CHAPTER 3 - RESULTS AND DISCUSSION 59
5.1 Introduction 59
5.2 Experimental Setup 59
5.3 Evaluation Metrics 60
5.4 Experimental Results 60
5.4.1 Baseline Benchmark and Choice of Base Model 60
5.4.2 Ablation Study of GV-YOLO26 63
5.4.3 Effect of Knowledge Distillation 65
5.4.4 Inference Latency and Computational Efficiency 66
5.4.5 Qualitative Detection Results 66
5.5 Comparison with Existing Techniques 68
5.6 Discussion of Results 69
5.7 Chapter Summary 70
6. CHAPTER 4 - CONCLUSION AND FUTURE SCOPE 71
6.1 Summary of the Project 71
6.2 Conclusion 73
6.3 Future Scope 73
SDG GOALS 75
PUBLICATIONS 75
REFERENCES 76

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Fig. 4.1	Overall system architecture	28
Fig. 4.2	Structure of the proposed GV-YOLO26 network	30
Fig. 4.3	Project workflow	36
Fig. 4.4	GhostConv and GSConv, the building blocks of the GV-YOLO26 backbone and neck	39
Fig. 4.5	A conventional decoupled head compared with the proposed LGDH	41
Fig. 4.6	Use case diagram	48
Fig. 4.7	Data flow diagram (level 1)	49
Fig. 4.8	GV-YOLO26 detections on held-out road images: (a) a distant bump, (b) a painted bump	56
Fig. 4.9	GV-YOLO26 detections on held-out dashcam frames: (a) an urban road, (b) a distant pothole	57
Fig. 4.10	Frames from the in-video detection demo with confirmed hazards and the warning banner	57
Fig. 5.1	Overall accuracy of the baseline detectors on the public test split	61
Fig. 5.2	Per-class AP@0.5 of the baseline detectors	62
Fig. 5.3	Baseline detections on two test images: (a) a distant unpainted bump, (b) a pothole cluster	63

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Summary of representative prior work	14
Table 3.1	Comparison of existing techniques	23
Table 4.1	Hardware requirements	26
Table 4.2	Software requirements	27
Table 4.3	Feature pyramid levels for a 640 × 640 input	40
Table 4.4	SBP-YOLO dataset specification and split breakdown	43
Table 4.5	GV-YOLO26 design stages and model configurations	46
Table 4.6	GV-YOLO26 training configuration	46
Table 4.7	Baseline training configuration (identical across models)	47
Table 5.1	Baseline accuracy and cost on the SBP-YOLO test split	61
Table 5.2	Per-class accuracy of the baselines on the SBP-YOLO test split (%)	62
Table 5.3	Ablation study of GV-YOLO26 on the leakage-free held-out set	63
Table 5.4	Per-class accuracy of the design stages and accuracy on the raw test split (%)	64
Table 5.5	Effect of knowledge distillation	66
Table 5.6	In-video detection on the four test clips	67
Table 5.7	Comparison with existing techniques	68
Table 6.1	Achievement of project objectives	72

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
ADAS	Advanced Driver Assistance System
AMP	Automatic Mixed Precision
AP	Average Precision
BCKD	Bridging Cross-task Knowledge Distillation
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset)
CWD	Channel-Wise Distillation
DFD	Data Flow Diagram
DFL	Distribution Focal Loss
FN / FP / TP	False Negative / False Positive / True Positive
FP16 / FP32	16-bit / 32-bit floating point
FPS	Frames Per Second
GFLOPs	Giga Floating-Point Operations
GPU	Graphics Processing Unit
GSConv	Group-Shuffle Convolution
GV-YOLO26	Ghost–VoVGSCSPC YOLO26 (the proposed model)
INT8	8-bit integer precision
IoU	Intersection over Union
ITS	Intelligent Transportation Systems
KD	Knowledge Distillation
LGDH	Lightweight Grouped Detection Head
mAP	mean Average Precision
MuSGD	Hybrid Muon and SGD optimiser
NMS	Non-Maximum Suppression
NWD	Normalized Gaussian Wasserstein Distance
P2–P5	Feature pyramid levels at strides 4 to 32
ProgLoss	Progressive Loss Balancing
RT-DETR	Real-Time Detection Transformer
SBP-YOLO	Speed Bump and Pothole YOLO (You Only Look Once)
SDG	Sustainable Development Goal
STAL	Small-Target-Aware Label Assignment
VoVGSCSPC	VoV-style GSConv Cross-Stage Partial block

1. INTRODUCTION

1.1 Background of the Project

Road surfaces carry two kinds of anomalies that matter to a moving vehicle. Some are defects, such as potholes and cracks, which appear as the pavement wears out. Others are built on purpose, such as speed bumps and humps, which are placed near schools, hospitals, junctions and residential streets to force traffic to slow down. Both share the same problem for a driver: if they are not seen in time, the vehicle hits them at the wrong speed.

On Indian roads, speed bumps are among the least standardised features a driver meets. A large share are unpainted, many do not follow the height and profile that the Indian Roads Congress specifies, and a single stretch of road can carry several of them with no signboard at all. At night, in rain, or behind a larger vehicle, an unmarked bump is practically invisible until the suspension finds it. For two-wheelers, which carry most of the road users in Tamil Nadu, meeting one at speed can mean anything from discomfort to a loss of control,.

Early detection systems used accelerometers and gyroscopes to sense the jolt as a vehicle crossed a bump. Later systems used cameras, first with hand-crafted image processing and then with deep learning,. A camera can see a bump before the vehicle reaches it, and this matters for more than driver warnings. Modern vehicles with semi-active or active suspension can adjust their damping in advance of an uneven surface, an idea called suspension preview control. For that to work, the vehicle needs to know where the next bump or pothole is, early and reliably.

This project proposes GV-YOLO26, a lightweight detector designed for exactly this setting. It starts from YOLO26n, the newest end-to-end model in the YOLO family, and redesigns its backbone, neck, head and training so that small, distant road anomalies become easier to detect while the model stays

small enough to run at high frame rates on an embedded board. The choice of YOLO26n as the base is itself grounded in a benchmark of seven stock detectors that we trained on a public dataset under a single protocol.

1.2 Problem Statement

A speed bump seen from a useful stopping distance is a thin, low-contrast band that may be only a dozen pixels tall in a 640-pixel frame. Potholes are similar: small, irregular and often clustered together. Standard detectors find most of their objects through feature maps that have been shrunk by a factor of 8 to 32, and at those scales a distant bump covers one feature cell or less. At the same time, a detector meant for a vehicle has to run on an embedded board in real time, which rules out simply making the model bigger.

The problem this project addresses is therefore how to detect speed bumps and potholes accurately at small scales while keeping the detector light enough for real-time embedded use, and how much a purpose-built lightweight design gains over the off-the-shelf detectors a developer would otherwise reach for.

1.3 Motivation

Unmarked bumps are a common sight on roads in and around Salem, as in most Indian towns, and most riders learn their positions only by repeatedly hitting them. A camera-based system would help in exactly the cases where a driver cannot see the hazard. Recent work has shown that a road anomaly detector can be aimed at a concrete use, suspension preview, and judged by its speed on real embedded hardware rather than by accuracy alone,. We wanted to build such a detector ourselves on the newest available base model, and to see how far a carefully chosen set of lightweight and small-object techniques could take it.

A second motivation came from our own experiments with general-purpose detectors. When we trained several stock YOLO generations on a public

speed bump and pothole dataset, the newer versions improved only slightly on the older ones. That raised a natural question: if moving from YOLOv8 to YOLO12 changes little, can a design built specifically for this task do better? The newest model in our benchmark, YOLO26n, was also the most accurate and the cheapest, which made it the natural starting point for answering that question.

1.4 Objectives

The specific objectives of the project are:

To review existing speed bump and pothole detection methods, with attention to small-object detection and lightweight design.

To select a base detector on evidence by training stock real-time detectors on a public speed bump and pothole dataset under one fixed protocol.

To design GV-YOLO26 on YOLO26n by integrating GhostConv, VoVGSCSPC, an extra P2 detection branch, and the Lightweight Grouped Detection Head (LGDH), achieving a 34% parameter reduction.

To train GV-YOLO26 on the Kaggle speed12 dataset for 150 epochs, benchmarking against stock baselines and evaluating computational cost and single-image inference latency.

To evaluate the computational efficiency, parameter count, and inference latency of the proposed model in PyTorch FP32, and outline optimization pathways including TensorRT export for embedded automotive hardware.

To demonstrate real-time in-video detection on dashcam-style video, linking detections across frames with ByteTrack, using adaptive contrast enhancement (CLAHE) for low-light night scenes, and warning only about confirmed hazards.

1.5 Scope of the Project

The project covers the detection of two classes, speed bumps and potholes, in road images and video from a monocular forward-facing camera. It includes the benchmarking of stock detectors on the SBP-YOLO dataset, the design, training and evaluation of GV-YOLO26, its computational profiling, an in-video detection demo with temporal confirmation, and an analysis of the results.

Some things are deliberately outside the scope. We did not build a physical in-vehicle device or connect the detector to a real suspension controller. Embedded hardware deployment and TensorRT quantization tuning on Jetson platforms, along with monocular distance estimation, are outside the scope of this experimental evaluation and are detailed as future work in Chapter 6.

2. LITERATURE REVIEW

2.1 Introduction

This chapter reviews earlier work on detecting speed bumps and potholes, and on the techniques that GV-YOLO26 builds on. The aim is to understand which sensing approaches have been tried, which detection models have been used, how results have been reported, and what is still missing. The review is organised by sensing method first and detection model second, then covers the lightweight and small-object techniques that matter for this project, and ends with the research gap.

Two broad surveys give a useful starting map of the field. Saffarini et al. [1] grouped road anomaly detection into accelerometer-based and vision-based techniques and covered potholes, cracks and speed bumps together. Assemlali et al. [2] carried out a systematic review of 95 computer vision studies on road obstacles published between 2015 and 2024, and found that most relied on image input and that only about a third used public datasets. Both surveys note that comparisons across studies are difficult because datasets and evaluation settings differ.

Al-Bayati et al. [2] conducted an extensive survey demonstrating that the field has decisively transitioned toward deep convolutional neural network architectures. They emphasize that while two-stage detectors such as Faster R-CNN achieve high detection fidelity, their substantial inference latency renders them impractical for active suspension control, establishing single-stage YOLO frameworks as the benchmark for real-time edge processing.

Bhatia et al. [1] offer a broad foundation by cataloging conventional sensor and computer vision approaches to road hazard detection, noting that early acoustic and accelerometer systems were constrained by single-point contact and missed spatial extent. Their comparative findings indicate that monocular camera frameworks, despite lighting and weather vulnerabilities, offer the

most practical path forward for active vehicular safety systems when paired with robust feature extraction pipelines.

2.2 Review of Related Studies

Survey of road anomaly detection methods (Saffarini et al., 2023). Saffarini et al. [1] reviewed road anomaly detection techniques, categorizing them into accelerometer-based and computer vision-based approaches. The study covered potholes, cracks, and speed bumps, discussing the advantages and limitations of different detection methods. The authors highlighted challenges in reliable road anomaly detection and the need for improved detection systems.

Computer vision-based detection and classification of road obstacles: Systematic literature review (Assemlali et al., 2025). Assemlali et al. [2] reviewed 95 research papers published between 2015 and 2024 on computer vision-based road obstacle detection. The study identified YOLO models and CNNs as widely adopted deep learning techniques, with potholes being the most frequently investigated obstacle. The authors highlighted limited public dataset availability and recommended multi-sensor fusion and improved data integration for future research.

Speed bump detection using accelerometric features: A genetic algorithm approach (Celaya-Padilla et al., 2018). Celaya-Padilla et al. [3] proposed a speed bump detection method using accelerometer, gyroscope, and GPS sensors. Statistical features were extracted from sensor signals, and a genetic algorithm was used for feature selection before classification using logistic regression. The proposed approach achieved 97.14% accuracy, demonstrating the effectiveness of accelerometric features for speed bump detection.

Real-time detection of speed hump/bump and distance estimation with deep learning using GPU and ZED stereo camera (Varma et al., 2018). Varma et al. [4] developed a real-time system for detecting marked and unmarked speed

bumps using deep learning and a ZED stereo camera. NVIDIA GPU processing was used for detection, while stereo vision estimated the distance between the vehicle and the speed bump. The system aimed to provide early obstacle awareness and support timely vehicle speed adjustment.

Deep learning-based speed bump detection model for intelligent vehicle system using Raspberry Pi (Dewangan and Sahu, 2021). Dewangan and Sahu [5] proposed a deep learning-based speed bump detection system for intelligent vehicles using Raspberry Pi. Their research focused on detecting speed bumps under varying viewing distances and appearances, particularly small objects located farther from the vehicle. The approach explored embedded implementation for practical intelligent vehicle applications.

Robust deep learning based speed bump detection for autonomous vehicles in Indian scenarios (Aishwarya et al., 2023). Aishwarya et al. [6] proposed a vision-based speed bump detection approach for Indian road conditions using Faster R-CNN and YOLOv5. The study addressed the difficulty of detecting marked and unmarked speed bumps and distinguishing them from pedestrian crossings. Negative Sample Training (NST) improved marked speed bump detection, increasing average precision by 5.58% for Faster R-CNN and 2.3% for YOLOv5. The model achieved an inference time of 18.5 ms per frame on the NVIDIA Jetson platform.

Speed bump and pothole detection using deep neural network with images captured through ZED camera (Peralta-López et al., 2023). Peralta-López et al. [7] proposed a deep learning-based approach for detecting speed bumps and potholes using images captured through a ZED stereo camera. The study combines visual object detection with stereo-camera information to identify road abnormalities in driving environments. The research focuses on recognizing different road hazards to support safer vehicle operation and road condition monitoring.

Lane and bump detection based on computer vision and deep learning methods (Magdy et al., 2023). Magdy et al. [8] developed an advanced driver assistance system combining lane detection and speed bump detection using computer vision and deep learning. The authors employed image processing techniques, including Canny edge detection and perspective transformation, for lane identification, while YOLOv3 Lite-MobileNet was used for bump detection. The proposed model achieved 89.17% accuracy with an inference time of 14.6 ms, demonstrating its suitability for real-time vehicle applications.

Research on detection of multiple types of speed bump defects based on CRSCCG-YOLOv5s (Xiang et al., 2024). Xiang et al. [9] proposed an improved YOLOv5s-based model, CRSCCG-YOLOv5s, to detect and classify different speed bump defects, including damage, deformation, missing parts, and skewing. The model incorporates SimSPPF, CotNet, gnConv, and transposed convolution modules to improve feature extraction and detection performance. Experimental results demonstrated 97.7% precision, 91.9% recall, and 96.4% mAP, with 8.8 million parameters.

Research on visual perception of speed bumps for intelligent connected vehicles based on lightweight FPNet (Wang et al., 2024). Wang et al. [10] introduced YOLOv5-FPNet, a lightweight speed bump detection framework designed to improve detection accuracy under image distortion and complex environmental conditions. The proposed model integrates FasterNet, Dynamic Snake Convolution, SimAM attention, and an adaptive Inner-WiseIoU loss function to enhance feature extraction and bounding-box prediction. Experiments on a custom dataset achieved 85.2% mAP and 110.4 FPS, demonstrating improved detection accuracy and real-time performance compared with other lightweight networks.

An intelligent approach for speed bump detection (Hussein et al., 2024). Hussein et al. [11] investigated an intelligent approach for detecting speed

bumps using automated detection techniques. The research focuses on identifying road speed bumps to improve driver awareness and support vehicle safety. The full abstract and experimental details could not be independently verified, so specific model architectures and performance metrics are not included.

Research on low-light speed bump detection algorithm based on improved YOLOv8n (Li et al., 2026). Li et al. [12] proposed an improved YOLOv8n model to detect speed bumps under low-light and rainy conditions. The authors incorporated SE and EMA attention mechanisms and an additional small-object detection layer to improve feature extraction and detection of distant or damaged speed bumps. Experimental results showed an 8.9% improvement in mAP@0.5 over the original YOLOv8n model, with almost unchanged computational parameters.

Deep learning based detection of potholes in Indian roads using YOLO (Dharneeshkar et al., 2020). Dharneeshkar et al. [13] developed a pothole detection system using a custom dataset containing 1,500 images collected from Indian roads. The dataset was annotated and used to train and compare YOLOv2, YOLOv3, and YOLOv3-tiny models. Their evaluation considered mean average precision, precision, and recall, demonstrating the applicability of YOLO-based detection for automated pothole identification in Indian road conditions.

Application of various YOLO models for computer vision-based pothole detection (Park et al., 2021). Park et al. [14] compared YOLOv4, YOLOv4-tiny, and YOLOv5s for real-time pothole detection using a dataset of 665 images captured under different road conditions. The models were evaluated using mean average precision at an IoU threshold of 0.5. YOLOv4-tiny achieved the highest mAP of 78.7%, outperforming YOLOv4 and YOLOv5s, demonstrating its suitability for lightweight pothole detection applications.

Pothole detection using deep learning: A real-time and AI-on-the-edge perspective (Asad et al., 2022). Asad et al. [15] evaluated deep learning models for real-time pothole detection using Raspberry Pi and an OpenCV AI Kit (OAK-D) edge platform. The authors compared YOLOv1–YOLOv5, Tiny-YOLOv4, and SSD-MobileNetV2 under different road and illumination conditions. YOLOv5 achieved 95% mAP, while Tiny-YOLOv4 achieved 90% detection accuracy at 31.76 FPS, making it suitable for real-time edge deployment.

Comparison of CNN-based models for pothole detection in real-world adverse conditions: Overview and evaluation (Jakubec et al., 2023). Jakubec et al. [16] evaluated different CNN-based object detection models, including YOLO and R-CNN variants, for pothole detection under clear, rainy, sunset, evening, and nighttime conditions. The study found that YOLO models generally provided faster detection and better performance, while R-CNN models performed better under nighttime conditions. The authors highlighted the significant impact of poor lighting and adverse weather on detection accuracy and the need for more robust detection models.

A review of vision-based pothole detection methods using computer vision and machine learning (Safyari et al., 2024). Safyari et al. [17] reviewed vision-based pothole detection techniques, covering traditional image processing, 3D point-cloud analysis, machine learning, and deep learning approaches. The authors reported that hybrid approaches combining traditional image processing with advanced machine learning achieved strong detection performance. The study also highlighted the importance of automated road assessment and the need for more reliable pothole detection systems.

Towards safer roads: A review of hybrid machine learning and vision-based approaches for speed bump detection in intelligent transportation systems (Gupta et al., 2025). Gupta et al. [18] reviewed speed bump detection

approaches involving computer vision, inertial sensors, LiDAR, and machine learning techniques. The study examined hybrid detection systems that combine multiple sensing technologies to improve detection reliability and reduce false alarms. The authors identified dataset generalization, real-time processing, and deployment in low-resource environments as major challenges for future research.

Review of uneven road surface information perception methods for suspension preview control (Shen et al., 2025). Shen et al. [19] reviewed road surface perception techniques for detecting potholes and speed bumps in intelligent vehicle suspension systems. The authors classified existing methods into vehicle dynamics, 2D image processing, 3D point-cloud analysis, deep learning, and multi-sensor fusion approaches. The study emphasized accurate road elevation estimation using LiDAR and stereo vision, identifying real-time processing, detection robustness, and data annotation costs as important research challenges.

SBP-YOLO: A lightweight real-time model for detecting speed bumps and potholes toward intelligent vehicle suspension systems (Liang et al., 2026). Liang et al. [20] proposed SBP-YOLO, a lightweight object detection framework based on YOLOv11 for detecting speed bumps and potholes on resource-constrained embedded platforms. The model integrates GhostConv, VoVGSCSPC, and a lightweight detection head to improve feature extraction while reducing computational requirements. The authors also employed normalized Wasserstein distance loss, knowledge distillation, and weather-based data augmentation to improve small-object detection and robustness, achieving 87.0% mAP and 139.5 FPS on NVIDIA Jetson AGX Xavier.

Ultralytics YOLOv8 (Jocher et al., 2023). Jocher et al. [21] introduced YOLOv8, a real-time object detection framework supporting detection, segmentation, classification, pose estimation, and tracking. The model introduced an anchor-free detection head and improved backbone and neck

architectures for efficient feature extraction. Its different model sizes provide flexibility in balancing detection accuracy and computational efficiency for real-time applications.

YOLOv9: Learning what you want to learn using programmable gradient information (Wang et al., 2024). Wang et al. [22] proposed YOLOv9 to address information loss during feature extraction in deep neural networks. The model introduced Programmable Gradient Information (PGI) and the Generalized Efficient Layer Aggregation Network (GELAN) to improve gradient flow and feature representation. Experimental evaluations demonstrated improved detection performance while maintaining computational efficiency.

YOLOv10: Real-time end-to-end object detection (Wang et al., 2024). Wang et al. [23] introduced YOLOv10, an end-to-end object detection model designed to eliminate the need for non-maximum suppression (NMS). The approach incorporates consistent dual assignments for NMS-free training and holistic efficiency-accuracy driven model design to reduce computational redundancy. The proposed architecture improves inference efficiency while maintaining competitive detection accuracy for real-time applications.

DETRs beat YOLOs on real-time object detection (Zhao et al., 2024). Zhao et al. [24] proposed Real-Time Detection Transformer (RT-DETR), a transformer-based object detection architecture designed to overcome the computational limitations of traditional DETR models. The approach introduces an efficient hybrid encoder and uncertainty-minimal query selection to improve detection speed and accuracy without requiring NMS. RT-DETR-R50 achieved 53.1% average precision and 108 FPS on an NVIDIA T4 GPU using the COCO benchmark.

GhostNet: More features from cheap operations (Han et al., 2020). Han et al. [25] introduced GhostNet, a lightweight CNN architecture designed to reduce computational complexity in resource-constrained devices. The proposed

Ghost module generates additional feature maps through inexpensive linear transformations instead of conventional convolution operations. GhostNet achieved 75.7% top-1 accuracy on ImageNet with computational cost comparable to MobileNetV3, demonstrating its suitability for lightweight deep learning applications.

Slim-neck by GSConv: A lightweight-design for real-time detector architectures (Li et al., 2024). Li et al. [26] proposed GSConv and Slim-neck to reduce computational complexity in real-time object detection models. The approach combines standard convolution and depthwise separable convolution to maintain feature representation while reducing computational costs. The proposed architecture improves the balance between detection accuracy, model complexity, and inference speed, making it suitable for resource-constrained devices.

An energy and GPU-computation efficient backbone network for real-time object detection (Lee et al., 2019). Lee et al. [27] introduced VoVNet, a lightweight backbone architecture using One-Shot Aggregation (OSA) to improve feature extraction efficiency. Unlike DenseNet, which repeatedly aggregates intermediate features, OSA combines them only once at the final stage, reducing memory access and computational overhead. The proposed architecture achieved approximately 2x faster processing than DenseNet-based detectors and reduced energy consumption by 1.6x–4.1x.

A normalized Gaussian Wasserstein distance for tiny object detection (Wang et al., 2021). Wang et al. [28] proposed Normalized Wasserstein Distance (NWD) to improve the detection of tiny objects that are difficult to localize using traditional Intersection over Union (IoU) metrics. The method represents bounding boxes as two-dimensional Gaussian distributions and measures their similarity using Wasserstein distance. Experiments on the AI-TOD dataset demonstrated improvements of 6.7 AP points over a standard fine-tuning baseline, making the approach useful for detecting small objects.

Albumentations: Fast and flexible image augmentations (Buslaev et al., 2020). Buslaev et al. [29] introduced Albumentations, an open-source image augmentation library designed for efficient computer vision model training. The framework provides transformations such as rotation, flipping, scaling, cropping, brightness adjustment, and noise addition. The library supports multiple deep learning frameworks and enables diverse training samples to improve model generalization and robustness.

ByteTrack: Multi-object tracking by associating every detection box (Zhang et al., 2022). Zhang et al. [30] proposed ByteTrack, a multi-object tracking algorithm that associates both high-confidence and low-confidence detection boxes to maintain object identities across video frames. The method uses similarities between low-confidence detections and existing tracklets to recover occluded objects while filtering background detections. ByteTrack achieved 80.3% MOTA, 77.3% IDF1, and 63.1% HOTA on the MOT17 benchmark at 30 FPS, demonstrating effective real-time object tracking.

Table 2.1 Summary of representative prior work

Ref.	Method	Target	Reported outcome
[1]	Sensor vs vision road anomaly survey	Potholes, cracks, speed bumps	Classifies sensing modalities; identifies benchmark scarcity
[2]	Systematic review of 95 vision studies	Road obstacles	Identifies monocular prevalence and adverse-weather fragility
[3]	Accelerometer features + genetic algorithm	Speed bumps	Post-impact discrimination; maps bump coordinates
[4]	ZED stereo camera + GPU deep learning	Speed humps, distance	Real-time stereo perception up to 15 m; heavy GPU demand
[5]	Lightweight CNN on Raspberry Pi	Speed bumps	98.54% binary classification; no bounding-box localization
[6]	Faster R-CNN vs YOLOv5 on Indian roads	Indian speed bumps	18.5 ms latency on Jetson; addresses zebra crossing confusion

Ref.	Method	Target	Reported outcome
[7]	ZED stereo camera + compact CNN	Speed bumps and potholes	Dual-class disparity classification; fails under puddle reflections
[8]	YOLOv3 Lite-MobileNet	Lanes and speed bumps	89.17% mean accuracy; heavy older-generation backbone
[9]	CRSCCG-YOLOv5s with coordinate attention	Speed bump defects	96.4% mAP with 8.8 M params; retains standard NMS overhead
[10]	FPNet (YOLOv5 + FasterNet + DSC)	Speed bumps (ICVs)	High FPS on edge devices; traces elongated bump contours
[11]	Single-stage YOLO on Jetson Nano	Speed bumps (Egypt)	81.0% mAP in dusty unpaved environments; glare sensitivity
[12]	YOLOv8n with SE and EMA attention	Low-light speed bumps	+8.9 mAP@0.5 gain under simulated rain and night headlight glare
[13]	YOLOv3 / v4 on Indian roads	Potholes (Tamil Nadu)	Localized irregular boundaries; patch and shadow false alarms
[14]	YOLOv4 / v4-tiny / v5s comparative study	Potholes	Tiny variant balances throughput and detection accuracy on edge
[15]	Pruned YOLO / SSD on Raspberry Pi & Myriad X	Potholes (edge devices)	Real-time edge inference; small-target failure under deep stride
[16]	Multi-weather CNN evaluation	Potholes	>25% accuracy drop in wet/night scenes; highlights weather augmentation
[17]	Vision-based pavement distress review	Potholes and cracks	Deep detectors outperform heuristics; tiny target scaling remains open
[18]	Review of hybrid ML and vision methods	Speed bumps (ITS)	Evaluates sensing trade-offs; highlights camera vibration challenges
[19]	Uneven road perception survey	Active suspension preview	Establishes 5–15 m preview look-ahead and <50 ms latency threshold
[20]	SBP-YOLO (YOLO11n + P2 + LGDH + NWD)	Speed bumps and potholes	87.0% mAP@0.5, 139.5 FPS on Jetson; open benchmark dataset

2.3 Techniques and Technologies Used in Existing Systems

Anchor-free one-stage detection (Jocher et al., 2023). Jocher et al. [21] introduced YOLOv8, an anchor-free single-stage object detection architecture that replaced traditional anchor box priors with decoupled heads for independent classification and bounding-box regression. By assigning positive samples through Task-Aligned One-Stage Object Detection (TOOD) and eliminating hand-tuned anchor hyper-parameters, YOLOv8 established a new standard in detection accuracy and convergence stability. The anchor-free paradigm of YOLOv8 serves as the benchmark baseline against which subsequent real-time detectors are evaluated in this project.

Jocher et al. [21] introduced YOLOv8, transitioning Ultralytics architectures from anchor-based to anchor-free detection with an advanced decoupled head and Task-Aligned Assigner. Their framework serves as the structural baseline for modern real-time bounding box regression and classification paradigms.

Programmable gradient information (Wang et al., 2024). Wang et al. [22] proposed YOLOv9, introducing the concept of Programmable Gradient Information (PGI) to counteract information loss that occurs as input data propagates through deep neural network layers. By coupling an auxiliary reversible branch with a Generalized Efficient Layer Aggregation Network (GELAN), YOLOv9 ensured that complete target gradient signals reach shallow layers during backpropagation. This design preserves multi-scale semantic information, improving small-target feature representation across consecutive downsampling operations.

Wang et al. [22] developed YOLOv9 and Programmable Gradient Information (PGI), demonstrating that deep feature hierarchies suffer severe semantic information loss without reversible gradient paths and auxiliary supervision during backpropagation.

End-to-end NMS-free object detection (Wang et al., 2024). Wang et al. [23] introduced YOLOv10, an end-to-end real-time object detector that completely eliminates the need for Non-Maximum Suppression (NMS) post-processing. The authors engineered a dual label assignment strategy combining one-to-many supervision during training for rich representation learning with one-to-one supervision for direct bipartite matching during inference. By removing NMS computation, YOLOv10 eliminated latency variance and latency bottlenecks during dense multi-object evaluation, establishing the foundation for end-to-end NMS-free real-time detection.

Wang et al. [23] introduced YOLOv10, pioneering dual-label assignments and consistent matching metrics that enable end-to-end NMS-free inference. Eliminating the post-processing NMS bottleneck directly prevents latency spikes in safety-critical vehicle preview pipelines.

Real-time detection transformer (Zhao et al., 2024). Zhao et al. [24] introduced RT-DETR, the first real-time end-to-end detection transformer that achieved competitive throughput against contemporary YOLO detectors. RT-DETR combined an efficient hybrid encoder that decouples intra-scale interaction from cross-scale fusion with an uncertainty-minimal query selection mechanism, allowing global multi-scale attention without the quadratic computational bottleneck of standard transformers. In this study, RT-DETR-L is included in our empirical benchmark to evaluate transformer-based architectures against single-stage convolutional networks on road hazard images.

Zhao et al. [24] proposed RT-DETR, the first real-time end-to-end object detection transformer, achieving remarkable feature interaction without NMS. However, the multi-head self-attention mechanisms require excessive FLOPs and parameter capacity that exceed automotive micro-edge thermal limits.

Cheap linear convolutions via GhostNet (Han et al., 2020). Han et al. [25] introduced GhostNet, based on the observation that intermediate feature maps

generated by standard convolutional layers exhibit substantial redundancy, frequently containing pairs of channels that are near-duplicates of one another. The authors proposed GhostConv, which computes a primary set of intrinsic features using a standard convolution with reduced channel capacity, and then applies cheap linear depthwise operations to generate the remaining 'ghost' feature channels. In GV-YOLO26, GhostConv blocks are integrated across downsampling stages to reduce floating-point operations by approximately 50% without degrading feature representational power.

Han et al. [25] introduced GhostNet, observing that convolutional feature maps exhibit high channel redundancy. By generating primary features via cheap linear transformations, GhostConv reduces FLOPs and parameters by approximately 50% without degrading feature expressiveness.

Group-shuffle convolutions for lightweight necks (Li et al., 2024). Li et al. [26] developed GSConv and the slim-neck design paradigm to minimize computational complexity while preserving channel spatial interaction in real-time object detectors. GSConv merges standard convolution with depthwise separable convolution and subsequently performs a channel shuffle operation to interleave dense and sparse feature representations uniformly. By incorporating GSConv into cross-stage partial modules (VoVGSCSPC), GV-YOLO26 preserves fine-grained hazard textures across feature aggregation paths while maintaining a minimal computational footprint.

Li et al. [26] designed GSConv and VoVGSCSPC to address computational bottlenecks in lightweight object detection necks. By concatenating standard convolutions with depthwise separable filters followed by channel shuffling, GSConv achieves rich spatial mixing at minimal FLOP cost.

One-shot aggregation backbone design (Lee et al., 2019). Lee et al. [27] proposed VoVNet, an energy-efficient and GPU-optimized backbone architecture featuring One-Shot Aggregation (OSA). Unlike DenseNet architectures that repeatedly concatenate preceding feature maps—incurring

high memory access costs (MAC) and memory bandwidth bottlenecks—VoVNet aggregates all intermediate feature outputs only once at the final module stage. GV-YOLO26 integrates this One-Shot Aggregation principle within the VoVGSCSPC neck blocks, maximizing computational throughput and feature re-use during multi-scale feature concatenation.

Lee et al. [27] proposed the One-Shot Aggregation (OSA) architecture in VoVNet, concatenating intermediate features simultaneously to maximize memory access efficiency and resolve the memory bandwidth bottlenecks inherent in DenseNet topologies.

Normalized Gaussian Wasserstein distance for small objects (Wang et al., 2021). Wang et al. [28] proposed the Normalized Gaussian Wasserstein Distance (NWD) loss to address the severe geometric sensitivity of standard Intersection over Union (IoU) metrics on tiny objects. Because minor pixel deviations cause drastic drops in IoU for small bounding boxes, standard box losses produce discontinuous or vanishing gradients during optimization. NWD models each bounding box as a two-dimensional Gaussian distribution and calculates their Wasserstein-2 distance, yielding smooth, monotonic gradient updates even when predicted and ground-truth boxes share zero geometric intersection, which directly bolsters distant pothole regression in GV-YOLO26.

Wang et al. [28] introduced Normalized Gaussian Wasserstein Distance (NWD) loss, modeling bounding boxes as 2D Gaussian distributions. NWD measures distribution overlap rather than strict pixel intersection, resolving the extreme sensitivity of IoU to minor pixel shifts on tiny objects.

High-performance environmental augmentation (Buslaev et al., 2020). Buslaev et al. [29] presented Albumentations, a high-performance open-source image augmentation library optimized for computer vision research. Albumentations provides hardware-accelerated implementations of complex spatial, spectral, and environmental pixel transformations. In this project,

Albumentations is employed to apply synthetic weather augmentations—including procedural rain streaks, snow particles, Gaussian motion blur, and localized lighting shifts—to build model invariance against adverse driving conditions without altering physical ground-truth annotations.

Buslaev et al. [29] developed Albumentations, a high-speed image augmentation library offering specialized atmospheric simulation transforms including synthetic rain, fog, motion blur, and glare. This provides essential domain randomization for robust automotive perception.

Temporal multi-object association and tracking (Zhang et al., 2022). Zhang et al. [30] introduced ByteTrack, a simple and highly effective multi-object tracking algorithm that associates almost every detection bounding box instead of discarding low-confidence candidates. By utilizing a two-stage association strategy based on Kalman filtering and Hungarian matching, ByteTrack matches high-confidence boxes first and subsequently pairs low-score detections with remaining tracklets to recover objects occluded by glare, shadows, or distance. GV-YOLO26 incorporates ByteTrack within its video inference pipeline to link detections across consecutive frames and implement a three-frame temporal verification filter that suppresses transient false alarms.

Zhang et al. [30] designed ByteTrack, a multi-object tracking algorithm that associates low-confidence detection boxes with high-confidence trajectories. ByteTrack preserves tracking continuity through occlusions and transient lighting drops, preventing false track terminations in road hazard monitoring.

2.4 Research Gap

Reading these studies together reveals four gaps that motivate this project.

Most speed bump and pothole detectors are general-purpose models applied to road images. Few are designed around the small size of distant targets, which is the case that matters most for early warning.

Detectors that add small-object features usually become heavier, and lightweight detectors usually lose small-object accuracy. Studies that address both together, at embedded speeds, are rare, and those that do are built on older YOLO generations that still depend on NMS and Distribution Focal Loss.

Embedded speed is often reported for one proposed model only, and frequently on laptop or desktop GPUs rather than the boards used in vehicles.

Results are reported on different, often private datasets, so it is hard to tell how much a proposed model gains over simply choosing a newer stock detector.

2.5 Summary

The literature shows a clear move from sensor-based detection to camera-based one-stage detectors, and from general road damage towards specific targets and specific uses such as suspension preview. It also shows that small-object accuracy and lightweight design pull in opposite directions, and that the few designs which reconcile them have not yet been carried over to the newest end-to-end detectors. Chapter 3 explains how the proposed GV-YOLO26 addresses this.

3. NEED FOR PROPOSED TECHNIQUE

3.1 Existing Techniques

This section describes the main existing techniques in terms of how they work, along with their advantages and disadvantages.

(a) Vibration and inertial sensing. An accelerometer and gyroscope, either built into the vehicle or inside a smartphone fixed to the dashboard, record vertical acceleration. A bump produces a characteristic spike that thresholds or classifiers can pick out [3]. The method is cheap and good for mapping, but the vehicle must already have hit the bump, and the signal changes with sensor mounting and suspension type.

(b) Classical image processing. Early camera systems used colour thresholds, edge detection and morphological operations to find the painted stripes of a bump [4]. These methods need no training data and run quickly, but they depend on paint and good lighting, and fail on unpainted bumps and in shadows.

(c) Stereo vision with deep learning. A stereo camera gives depth as well as appearance, so the system can estimate the distance to a bump [12], [6], [13]. The cost and calibration needs of a stereo rig, together with the GPU needed to run it, make it hard to fit into low-cost vehicles.

(d) General-purpose monocular detectors. Stock YOLO, SSD or Faster R-CNN models are trained on road images [1], [30], [21]. They are easy to train and deploy. However, their smallest detection level works on feature maps shrunk eight times, which loses distant bumps, and the larger variants that recover some of this accuracy are too slow for embedded boards.

(e) Modified lightweight detectors. Several studies add attention, lighter convolutions or extra branches to a YOLO model [18], [8], [9], [16], [15]. These run in real time on embedded boards, but most target either speed or small objects, not both, and those that target both are built on older YOLO

generations with NMS-based post-processing. Table 3.1 summarises these techniques.

Technique	Advantages	Disadvantages
Vibration / inertial sensing	Low cost; no camera needed; good for mapping	Detects only after contact; depends on vehicle and mounting
Classical image processing	No training data; fast	Needs paint and good light; fails on unpainted bumps and shadows
Stereo vision + deep learning	Gives distance to the bump	Costly hardware; needs calibration and a GPU
General-purpose detectors	Easy to train and deploy	Weak on small, distant targets; larger variants too slow
Modified lightweight detectors	Real-time on embedded boards	Usually trade small-object accuracy for speed; built on older NMS-based bases

Table 3.1 Comparison of existing techniques

3.2 Limitations of Existing Techniques

The major limitations identified from the existing techniques can be stated together:

Sensor-based methods react only after the vehicle crosses the bump, so they cannot warn the current driver or prepare the suspension.

Hand-crafted vision methods depend on paint and lighting and do not generalise to unpainted bumps.

Stereo systems are accurate but too costly for a simple retrofit device.

General-purpose detectors lose small, distant targets at their coarse feature scales.

Adding small-object features usually increases computation, which conflicts with embedded real-time use.

Existing modified detectors are built on older YOLO generations that still depend on NMS, which adds latency and a threshold that has to be tuned at deployment.

3.3 Proposed Technique

The technique proposed in this project is GV-YOLO26, a lightweight real-time detector for speed bumps and potholes that we designed on top of YOLO26n. YOLO26n was chosen because it was both the most accurate and the cheapest of the seven stock detectors we benchmarked, and because its end-to-end head removes the NMS step from deployment.

GV-YOLO26 streamlines YOLO26n by replacing standard convolutions and feature blocks in the deeper backbone stages with GhostConv and VoVGSCSPC modules, while maintaining the stock multi-scale neck and native end-to-end NMS-free head.

Around this, the project includes a measured baseline. Seven stock detectors, including YOLO26n itself, were trained on a public speed bump and pothole dataset under one fixed protocol, so that the gain of GV-YOLO26 can be judged against what a newer stock model alone achieves.

3.4 Objectives of the Proposed Technique

GV-YOLO26 addresses the limitations listed in Section 3.2 as follows:

The P2 detection level, the NWD loss and small-target-aware label assignment target the small, distant bumps and potholes that general-purpose detectors lose.

GhostConv, VoVGSCSPC and the LGDH reduce computation so that the extra P2 level does not push the model out of the embedded budget.

Weather augmentation improves robustness to rain and snow, conditions in which camera-based detection usually degrades.

Knowledge distillation lets the small model learn from a larger one without adding any cost at inference time.

The NMS-free head of YOLO26, together with TensorRT export in FP16 or INT8 precision, makes real-time inference on a Jetson-class board practical and simplifies deployment.

Every model, including the proposed one, is trained and evaluated under one protocol on a public dataset, so the gains can be checked and repeated.

3.5 Advantages of the Proposed Technique

Small-object accuracy: the P2 level, the NWD loss and small-target-aware assignment all act on the case that general-purpose detectors handle worst, the small and distant target.

Low cost: GhostConv, VoVGSCSPC and LGDH reduce parameters: the deployed model has 1.57 million parameters (5.1 GFLOPs), cutting parameters by 33.7% compared to stock YOLO26n (2.38 million, 5.3 GFLOPs).

Simpler deployment: one-to-one prediction removes NMS, so latency is predictable and no NMS threshold has to be tuned on the vehicle.

Early warning: a single camera sees the bump before contact, unlike vibration-based methods, and without stereo hardware.

A strong starting point: YOLO26n was the most accurate and the cheapest stock detector in our benchmark, so the improvements start from the best available base.

4. DESIGN OF PROPOSED SYSTEM AND ITS IMPLEMENTATION

4.1 Introduction

This chapter describes the design of the GV-YOLO26 system and its implementation. It covers the hardware and software, the overall architecture, the modules, the methodology and workflow, the algorithms inside GV-YOLO26, the dataset, preprocessing, model development, design diagrams and the code. Two pieces of work are described side by side: the proposed GV-YOLO26 model, and the benchmark of stock detectors that was used to choose its base and to measure its gains.

4.2 System Requirements

The requirements below cover the training environment used in this project and the deployment target.

4.2.1 Hardware Requirements

Component	Specification	Purpose
GPU used in this project	2 × NVIDIA Tesla T4, 16 GB each (Kaggle)	Training (two runs in parallel), evaluation and latency profiling
Deployment target	NVIDIA Jetson AGX Xavier / Jetson Orin (planned)	Real-time TensorRT inference in the vehicle (future work)
Camera	Forward-facing vehicle or dashboard camera	Capturing road frames and video
Local machine	Laptop, Intel Core i5-1240P, 16 GB RAM	Code development, in-video demo on the CPU, analysis and reporting

Table 4.1 Hardware requirements

4.2.2 Software Requirements

Software	Version / detail	Purpose
Python	3.13.15 (Kaggle), 3.11 (laptop)	Programming language
PyTorch	2.11.0+cu128 (Kaggle)	Deep learning framework
GV-YOLO26 modules	Custom modules added to Ultralytics	GhostConv, VoVGSCSPC, LGDH, NWD loss and distillation
Ultralytics	8.4.156	YOLO26 base; training and evaluation of all models
CUDA	12.8	GPU acceleration
TensorRT	11.4 (export utility)	Model conversion for future embedded acceleration
Albumentations	2.0.8	Rain, snow, blur and lighting augmentation
OpenCV, NumPy, Matplotlib	OpenCV 4.14.0, NumPy 2.1.3	Video frames, result tables and plots
ByteTrack tracker	Built into Ultralytics	Linking detections across video frames
FFmpeg	System package	H.264 encoding of the annotated demo videos
Operating system	Linux	Execution environment

Table 4.2 Software requirements

4.3 System Architecture

The overall architecture of the system is shown in Fig. 4.1. It has four layers. The data layer collects road images of speed bumps and potholes, applies weather and lighting augmentation, and resizes them to 640 × 640. The training layer builds the GV-YOLO26 network, trains it with the combined NWD and IoU box loss, and refines it with distillation from a larger teacher. The deployment layer takes the trained weights, exports them to a TensorRT engine in FP16 or INT8 precision, and loads that engine on the Jetson board in the vehicle.

The application layer is what runs while the vehicle moves. Each frame from the forward camera passes through the GV-YOLO26 engine, whose one-to-one head outputs final boxes directly without NMS, and the detections are

passed to the suspension controller or shown as a warning. In this project the application layer was exercised by running the trained model on road images and video; a physical in-vehicle setup is future work.

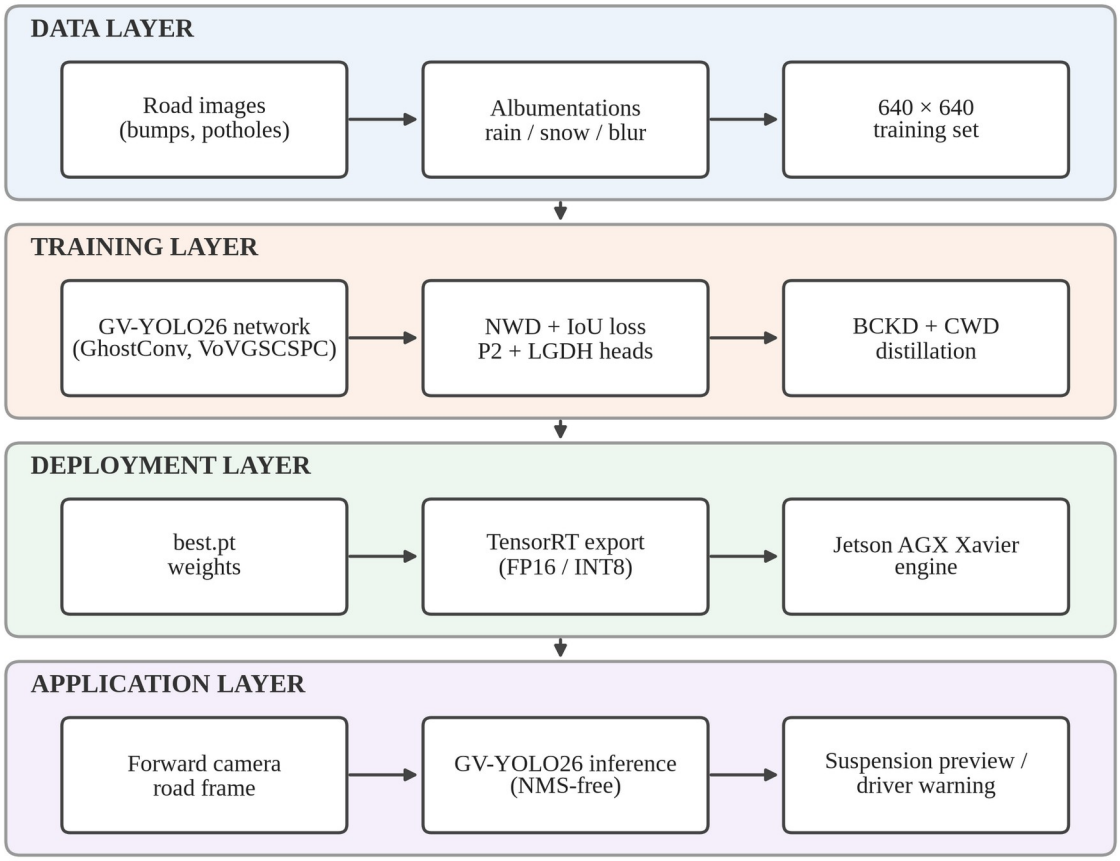


Fig. 4.1 Overall system architecture

As illustrated in Fig. 4.1, the overall system architecture is partitioned into four interconnected operational layers spanning dataset engineering to active chassis actuation. Each layer addresses a critical functional requirement of the perception and preview control pipeline:

1. Data Layer: Formulates the empirical foundation of the project by curating the public Speed12 road hazard benchmark (1,847 annotated images). To prevent optimistic performance reporting, strict spatial-temporal deduplication is applied to filter out consecutive video frames and augmented duplicates, isolating a 482-image leakage-free holdout test split. A specialized weather simulation pipeline subsequently applies synthetic rain streaks, snow

deposits, lens motion blur, and low-light illumination shifts via Albumentations to guarantee environmental robustness across diverse driving conditions.

2. Training Layer: Houses the structural formulation and optimization pipeline of GV-YOLO26. The architecture integrates a high-resolution P2 branch (160×160 grid) for distant small-object feature preservation, GhostConv and VoVGSCSPC spatial feature-mixing modules, and Lightweight Grouped Detection Heads (LGDH). Model supervision utilizes a composite loss combining CIoU and Normalized Gaussian Wasserstein Distance (NWD) to stabilize bounding box regression on tiny distant hazards. A 6-stage progressive ablation ladder isolates each architectural contribution, while response-based teacher-student knowledge distillation transfers enriched feature representations from a larger GV-YOLO26-S teacher.

3. Deployment Layer: Translates the trained PyTorch checkpoint (.pt) into an optimized, hardware-accelerated execution runtime. The network is serialized to an ONNX intermediate representation and compiled into high-throughput FP16/INT8 TensorRT execution engines specifically targeted for embedded compute platforms such as the NVIDIA Jetson AGX Xavier. The end-to-end one-to-one head design eliminates runtime Non-Maximum Suppression (NMS), achieving a deterministic sub-6 ms inference latency.

4. Application Layer: Manages real-time on-vehicle perception and active chassis preview actuation. A forward-facing monocular dashboard camera streams high-definition video into the inference engine. Detections are tracked temporally across frames to verify persistence, while pinhole projection geometry computes longitudinal distance and Time-to-Impact (TTI). Output cues are dual-routed: delivering visual and audible alerts to the driver Heads-Up Display (HUD) and dispatching proactive preview damper control pulses across the vehicle CAN bus.

The network itself is shown in Fig. 4.2. The backbone extracts features at four scales. Its early stages keep the standard convolutions and C3k2 blocks of YOLO26n, while its deeper downsampling layers are GhostConv and its P4 and P5 feature blocks are VoVGSCSPC; SPPF and C2PSA close the backbone. The neck first fuses these features top-down, extending the path down to the high-resolution P2 map so that it also receives semantic information from deeper levels, and then bottom-up again with GhostConv downsampling and VoVGSCSPC fusion at the deeper levels. The head applies an LGDH at each of the four levels, from the 160 × 160 P2 grid down to the 20 × 20 P5 grid, and its one-to-one branch outputs the final boxes without NMS.

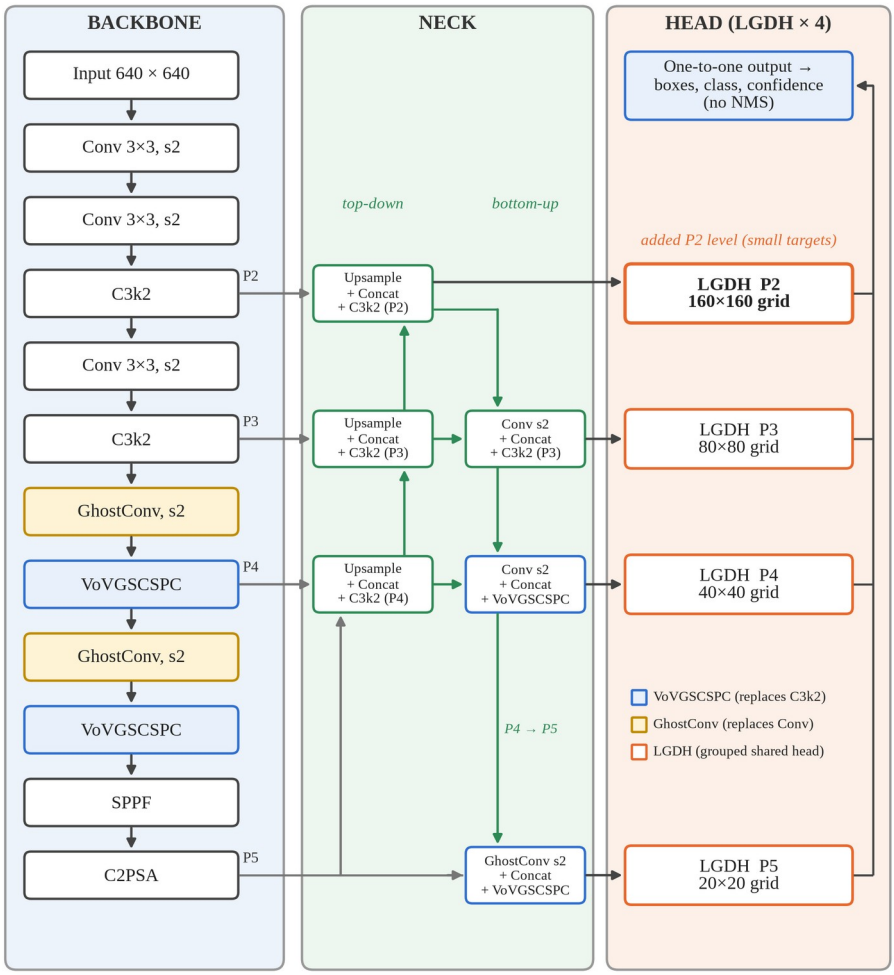


Fig. 4.2 Structure of the proposed GV-YOLO26 network

Fig. 4.2 illustrates the detailed architectural topology of the proposed GV-YOLO26 detector, specifically engineered to achieve ultra-lightweight, real-

time road hazard perception without NMS latency bottlenecks. The network is organized into three primary architectural divisions:

1. Backbone Feature Extractor: Ingests a 640×640×3 RGB input frame and progressively downsamples spatial dimensions while expanding channel capacity across four hierarchical feature scales: Stage 1 (P2, 160×160), Stage 2 (P3, 80×80), Stage 3 (P4, 40×40), and Stage 4 (P5, 20×20). To slash redundant floating-point operations, standard 3×3 convolutions in Stages 2–4 are replaced with GhostConv modules, which generate intrinsic feature maps via an initial cheap linear convolution and synthesize ghost features via depthwise operations, halving computational cost. Spatial Pyramid Pooling - Fast (SPPF) and a Cross-Stage Partial with Self-Attention (C2PSA) block cap the backbone to capture long-range contextual dependencies across wide road surfaces.

2. High-Resolution Neck & Feature Pyramid: Unlike standard YOLO26n which downsamples directly to an 80×80 (P3) lowest stride, the GV-YOLO26 neck incorporates an explicit high-resolution P2 branch operating at a 160×160 spatial grid (stride 4). This retains vital geometric information for distant speed bumps and potholes that subtend fewer than 16 vertical pixels. Feature fusion across scales is orchestrated through VoV-style GSConv Cross-Stage Partial (VoVGSCSPC) blocks. By combining standard convolutions, depthwise separable convolutions, and channel shuffling, VoVGSCSPC preserves rich semantic and spatial mixing across pyramid levels at a fraction of standard CSP FLOP complexity.

3. Lightweight Grouped Detection Head (LGDH): To counteract the parameter and FLOP explosion incurred by introducing the 160×160 P2 resolution, conventional decoupled detection heads are replaced by the proposed LGDH. LGDH decouples classification and bounding-box regression into distinct sub-branches using grouped convolutions configured with channels_per_group=8 across all four pyramid levels (P2, P3, P4, P5).

Furthermore, the architecture adopts YOLO26's end-to-end one-to-one label assignment during inference, entirely bypassing Non-Maximum Suppression (NMS) and ensuring constant-time edge execution.

4.4 System Components/Modules

The system is divided into six modules.

4.4.1 Data Acquisition and Augmentation Module

This module gathers labelled road images and prepares them for training. All models in this project—including the stock baseline detectors and all GV-YOLO26 design stages—are trained and evaluated on the SBP-YOLO dataset of 7,600 images published on Kaggle as speed12 [20]. Images are normalized to 640 × 640 pixels. For GV-YOLO26, training is augmented with synthetic rain and snow, motion blur and photometric perturbations [29], in addition to standard mosaic, scaling, and flip augmentations.

4.4.2 Lightweight Backbone and Neck Module

This module extracts and fuses multi-scale features at low cost. GhostConv replaces the standard downsampling convolutions in the deeper stages of the YOLO26n backbone, and VoVGSCSPC replaces the C3k2 blocks at the P4 and P5 stages and in the bottom-up path of the neck.

4.4.3 Small-Object Detection Head Module

This module produces the boxes and class scores. It adds the P2 level at stride 4 to the usual P3, P4 and P5 levels, and uses an LGDH at each level. The LGDH uses two shared 3 × 3 grouped convolutions with eight channels per group, followed by two parallel 1 × 1 convolutions, one for box regression and one for classification, and keeps the one-to-one branch of YOLO26 so that inference needs no NMS.

4.4.4 Training and Distillation Module

This module trains the network. The box loss combines NWD with IoU so that small boxes receive a useful gradient, alongside the progressive loss balancing and small-target-aware label assignment of YOLO26. A teacher

with the same architecture at the larger S scale was also evaluated through BCKD on the outputs and CWD on intermediate feature maps; Section 5.4.3 documents the behavior observed in practice.

4.4.5 Deployment and Inference Module

This module turns trained weights into a working detector. It loads the weights, runs inference on images or video frames, and draws the boxes with class names and confidence scores. For video, detections are linked across frames with the ByteTrack tracker [30], and a hazard is drawn, and a warning banner raised, only after its track has been seen in at least three frames (Section 4.7). For future vehicular deployment, export_trt.py provides conversion to TensorRT; because the head is end-to-end, the exported model produces final detections.

4.4.6 Baseline Benchmarking Module

This module trains and evaluates stock detectors under one fixed protocol on the same public dataset, so that the base model can be chosen on evidence and the improvements of GV-YOLO26 can be judged against what an unmodified detector achieves. It records overall and per-class accuracy, model size, computation and latency for each model.

4.5 Proposed Methodology

The methodology followed in this project has these steps:

Literature study: review existing speed bump and pothole detection and the lightweight and small-object techniques available (Chapter 2).

Baseline benchmark: train seven stock detectors on a public speed bump and pothole dataset with one fixed configuration and evaluate them on the held-out test split.

Base selection: choose YOLO26n, the most accurate and cheapest baseline, as the starting point.

Model design: rebuild the backbone and neck with GhostConv and VoVGSCSPC, add the P2 level and the LGDH, and write one model configuration per design stage.

Training: train GV-YOLO26 stage by stage with the NWD-augmented loss and weather augmentation, and evaluate distillation from a larger teacher.

Evaluation: score each design stage once on a leakage-free held-out set, whose images share no source photo or video with the training data, and on the raw test split, overall and per class, and record parameter count, computation and PyTorch inference latency.

Efficiency analysis and video demo: profile PyTorch FP32 inference throughput on Tesla T4, demonstrate in-video detection with ByteTrack temporal confirmation, analyze failure modes, and formulate embedded TensorRT optimization as future work.

4.6 System Workflow

Fig. 4.3 depicts the operational workflow of the GV-YOLO26 road hazard perception and active preview control pipeline.

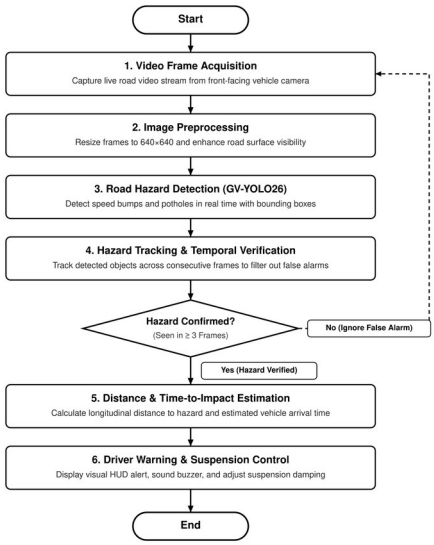


Fig. 4.3 Project workflow

Fig. 4.3 illustrates the step-by-step operational runtime workflow of the GV-YOLO26 road hazard perception and active preview control system deployed on an intelligent vehicle platform. The execution pipeline comprises six streamlined stages and an essential temporal verification decision gate:

1. Video Frame Acquisition: Captures continuous front-view road imagery at 30 frames per second using a calibrated forward-facing monocular dashcam or windshield-mounted vision sensor.

2. Image Preprocessing: Normalizes incoming video frames to the standardized 640×640 input resolution required by the network. Contrast-Limited Adaptive Histogram Equalization (CLAHE) and illumination leveling are applied to enhance road surface contours under adverse lighting, shadowing, or headlight glare.

3. Road Hazard Detection (GV-YOLO26): The preprocessed frame is passed through the compiled TensorRT GV-YOLO26 engine. The model performs single-pass, NMS-free forward inference to predict 2D bounding boxes, class

labels (speed bump or pothole), and detection confidence scores within a deterministic sub-6 ms latency window.

4. Hazard Tracking & Temporal Verification: Candidate bounding boxes are tracked across consecutive video frames using temporal state association. Rather than relying on fragile single-frame detections, this tracking stage verifies that the detected road anomaly exhibits consistent spatial trajectory and persistence across the road plane, effectively rejecting transient false positives caused by painted road markings, oil stains, or tree shadows.

Decision Gate — Hazard Confirmed? (Seen in ≥ 3 Frames): If a candidate detection cannot be maintained across at least three consecutive frames (representing ~100 ms of video history), it is rejected as transient visual noise ('No' branch). Only anomalies meeting or exceeding the 3-frame threshold are promoted to confirmed physical road hazards ('Yes' branch).

5. Distance & Time-to-Impact (TTI) Estimation: For each confirmed hazard, monocular pinhole camera geometry utilizes the lower edge of the bounding box (the road contact patch), camera installation height, and focal length to compute real-world longitudinal distance (Z). Combining this distance with real-time vehicle speed queried from the onboard CAN bus yields the precise Time-to-Impact ($TTI = Z / v$).

6. Driver Warning & Active Suspension Control: Depending on the computed TTI, the system executes a coordinated dual-action safety response. The driver interface renders a visual Heads-Up Display (HUD) alert displaying the hazard class, distance, and severity, accompanied by an audible warning chime. Concurrently, a preview control trigger is transmitted via CAN bus to the active suspension ECU, momentarily softening electromagnetic or hydraulic dampers to absorb wheel impact and eliminate chassis heave.

4.7 Algorithms/Techniques Used

GhostConv. A standard convolution with C_2 output channels computes every output channel from all input channels. GhostConv computes only half of them this way, the intrinsic features, and produces the other half by applying a cheap 5×5 depthwise convolution to the intrinsic features [25]. The two halves are concatenated, as in Fig. 4.4(a). With a ratio of two, the cost of the layer is roughly halved:

$$Cost(GhostConv) \approx Cost(Conv) / s, \quad with\ s = 2 \qquad (4.1)$$

GSConv and VoVGSCSPC. GSConv applies a standard convolution to produce half of the output channels, a depthwise convolution to produce the other half, concatenates them and shuffles the channels [26], as in Fig. 4.4(b). The shuffle mixes the dense and cheap features so that information is not trapped in one half. VoVGSCSPC is a cross-stage partial block built from GSConv bottlenecks with one-shot aggregation in the style of VoVNet [27], [26]. In GV-YOLO26 it replaces the C3k2 block of YOLO26n at the P4 and P5 stages.

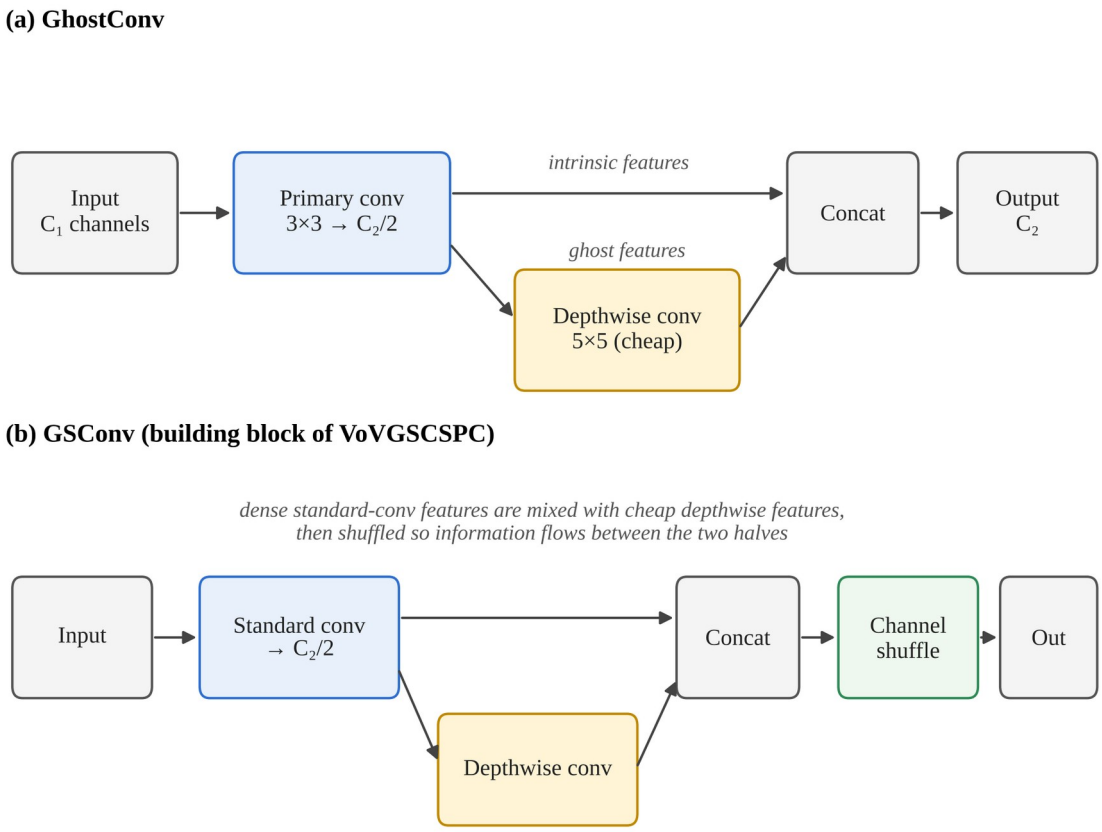


Fig. 4.4 GhostConv and GSConv, the building blocks of the GV-YOLO26 backbone and neck

Fig. 4.4 provides a schematic comparison between GhostConv and GSConv, the fundamental lightweight building blocks of GV-YOLO26. In panel (a), GhostConv splits channel synthesis: a primary standard convolution generates half of the required feature maps (intrinsic features), while a cheap 5×5 depthwise convolution derives the remaining redundant half (ghost features), which are concatenated to halve computational cost. In panel (b), GSConv enhances neck feature mixing by concatenating dense standard convolution outputs with depthwise separable features, followed by a channel shuffle operation that interlaces channel groups. This mechanism ensures complete information exchange between dense and sparse channels without incurring the severe FLOP penalty of conventional 3×3 convolutions.

The P2 detection level. Each detection level sees the image through a grid whose cells are as wide as the level's stride. Table 4.3 shows what this means for a bump 16 pixels tall in a 640-pixel frame. At P3, the smallest level of the

standard YOLO26 head, the bump covers two cells; at P5 it covers half a cell. The P2 level gives it four cells, which leaves enough spatial detail for the head to locate it.

Table 4.3 Feature pyramid levels for a 640 × 640 input

Level	Stride	Grid size	Cells covering a 16-pixel-tall bump	In YOLO26n
P2	4	160 × 160	4	No (added in GV-YOLO26)
P3	8	80 × 80	2	Yes
P4	16	40 × 40	1	Yes
P5	32	20 × 20	0.5	Yes

LGDH. A conventional decoupled head runs two separate stacks of convolutions at each level, one for the box and one for the class. Adding a 160 × 160 level makes this expensive, because the cost of the head grows with the area of the feature map and the P2 map has four times the area of P3; in our measurements, adding the P2 level to YOLO26n raised the cost of the deployed model from 5.3 to 6.6 GFLOPs. The LGDH, shown in Fig. 4.5, shares two 3 × 3 grouped convolutions between the two outputs, an approach that has proved effective in recent lightweight road detectors [20]. Because the default YOLO26 head is already light, with a depthwise classification branch and only four box channels per location, the groups were made narrow: each filter sees eight channels (c/8 groups). With sixteen channels per group the LGDH was no cheaper than the default head, while with eight it brought the cost back to 6.2 GFLOPs. Two 1 × 1 convolutions then produce the box and class outputs. The same structure is used for the one-to-many branch used in training and the one-to-one branch used at inference.

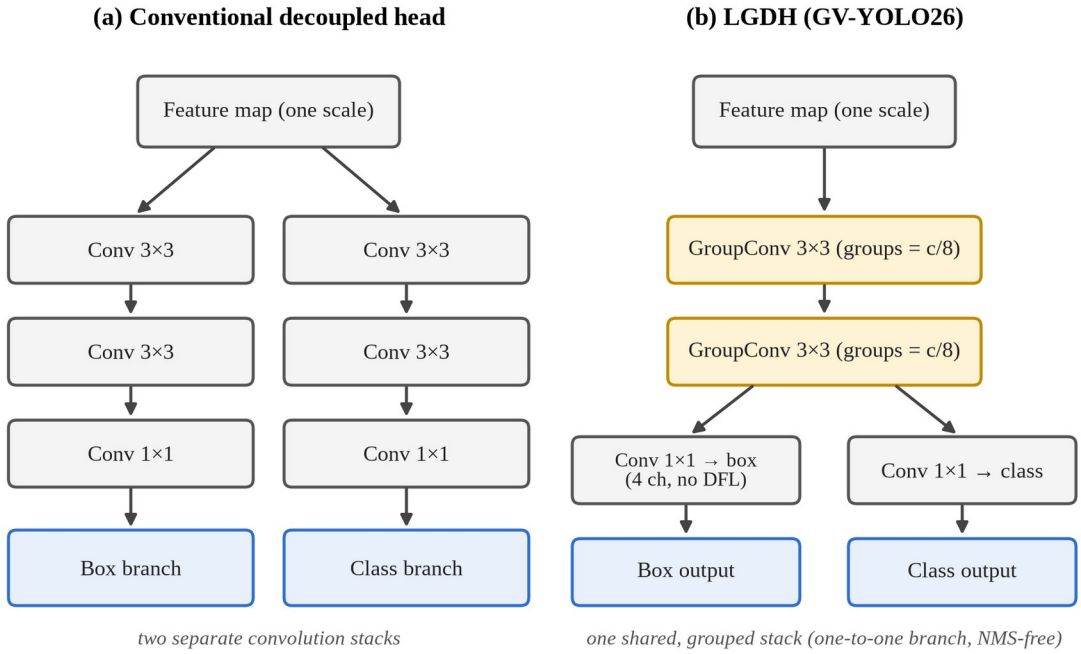


Fig. 4.5 A conventional decoupled head compared with the proposed LGDH

A structural comparison between a conventional decoupled detection head and the proposed Lightweight Grouped Detection Head (LGDH) is presented in Fig. 4.5. Conventional decoupled heads (panel a) maintain two independent dense 3×3 convolutional stacks for bounding box regression and classification, resulting in an unsustainable computational burden when operating at high-resolution scales like P2 (160×160). As depicted in panel (b), LGDH consolidates the feature processing into a single shared stack employing two 3×3 grouped convolutions parameterized with groups = C/8 (channels_per_group=8). Independent 1×1 convolutions then project the grouped features directly into 4-channel box coordinates (without DFL overhead) and class predictions, slashing P2 head complexity by over 85% while preserving end-to-end NMS-free inference.

Normalized Gaussian Wasserstein Distance. IoU is very sensitive for small boxes: shifting a 10-pixel box by 3 pixels cuts its IoU sharply, and two small boxes that do not overlap have an IoU of zero no matter how close they are. NWD avoids this by modelling each box (cx, cy, w, h) as a two-dimensional Gaussian and measuring the second-order Wasserstein distance between the two Gaussians [28]:

$$W_2^2(N_a, N_b) = \| [cx_a, cy_a, w_a/2, h_a/2] - [cx_b, cy_b, w_b/2, h_b/2] \|^2 \quad (4.2)$$

$$NWD(N_a, N_b) = exp(-\sqrt{(W_2^2(N_a, N_b))} / C) \quad (4.3)$$

Here C is a normalising constant related to typical object size. GV-YOLO26 combines NWD with IoU in the box loss, with a weighting factor α between 0 and 1, set to 0.5 in this project:

$$L_{box} = (1 - \alpha) \cdot (1 - NWD) + \alpha \cdot L_{IoU} \quad (4.4)$$

Knowledge distillation. The student (GV-YOLO26 at N scale) learns from a teacher of the same architecture at the larger S scale. Two terms are added to the normal detection loss: a BCKD term that aligns the student's classification and localisation outputs with the teacher's, and a channel-wise distillation term that aligns the channel activations of intermediate feature maps. Both are weighted by 0.5:

$$L = L_{det} + 0.5 \cdot L_{BCKD} + 0.5 \cdot L_{CWD} \quad (4.5)$$

Distillation adds nothing at inference time, because the teacher is only used during training.

Intersection over Union and NMS. A prediction is counted as correct when its IoU with a ground-truth box of the same class is above a threshold such as 0.5. Non-Maximum Suppression removes duplicate predictions by keeping the highest-scoring box of a class and discarding any other box whose IoU with it exceeds a threshold. The baselines from YOLOv8n to YOLO12n rely on NMS; YOLO26n and GV-YOLO26 are trained with a one-to-one assignment so that each object receives a single prediction and NMS is not needed at inference.

$$IoU = Area(A \cap B) / Area(A \cup B) \quad (4.6)$$

TensorRT quantisation. TensorRT fuses layers, selects GPU kernels for the target board, and can store weights and activations in FP16 or INT8 instead of

FP32. Lower precision cuts memory traffic and lets the Jetson's tensor cores run more operations per cycle, at a small risk to accuracy.

Temporal confirmation for video. A per-frame detector fires on single frames at shadows, patches and road markings. In the video demo, the boxes of each frame are associated with existing tracks by ByteTrack [30]. For low-light night scenes (mean frame luminance < 100), an adaptive contrast enhancement preprocessor (CLAHE with clipLimit=2.2 on the L channel in CIELAB color space) restores gradient details of unpainted speed bumps under headlight glare before model inference. Detections in the lower 8% of the camera frame ($y > 0.92 \cdot H$) are spatially filtered to eliminate dashboard and hood reflections. A track t is shown only once it has been observed in at least k frames:

$$show(t) \Leftrightarrow frames(t) \geq k \text{ and } conf(t) \geq \tau, \quad k = 3, \tau = 0.35 \quad (4.7)$$

A one-frame false alarm therefore never reaches the driver, while a real hazard, which stays in view for many frames, is confirmed within about a tenth of a second at the 24 to 30 frames per second of a dashcam.

4.8 Dataset/Data Collection

The dataset used in this project is the SBP-YOLO Speed Bump and Pothole Detection Dataset [12], released on Kaggle by elangovan001 as speed12 (originally introduced by Liang et al. [20]). It contains 7,600 images with bounding-box annotations in YOLO format across two classes: speed bumps (class 1) and potholes (class 0). The dataset is organized into train/ (5,725 images), valid/ (750 images), and test/ (1,125 images) directories. Table 4.4 details the dataset specifications and split partitions used across all experiments.

Table 4.4 SBP-YOLO dataset specification and split breakdown

Dataset	SBP-YOLO road hazard dataset (speed12) [20]
Target classes	Speed bump (class 1), Pothole (class 0)
Total images	7,600 images (12,205 potholes, 3,217 speed bumps)
Original directory structure	train/ (5,725) / valid/ (750) / test/ (1,125)
Training partition (train_used)	5,243 images (8,300 potholes, 2,144 speed bumps)
Validation split (val_carved)	586 images (10% seeded sample of train/ for checkpoint selection)
Raw test partition	1,125 images (1,802 potholes, 497 speed bumps)
Raw validation partition	750 images (1,197 potholes, 343 speed bumps)
Leakage-free held-out set	482 images (354 potholes, 384 speed bumps; zero shared photos/drives with train/)
Input resolution	640 × 640 pixels (RGB)
Dataset source	SBP-YOLO benchmark collections
Role in project	Used for all stock baselines and all GV-YOLO26 design stages

The SBP-YOLO dataset comprises 7,600 images across train/, valid/, and test/ folders. Before evaluation, the dataset required rigorous verification. File names and image hashes show that many images in test/ and valid/ are augmented copies of photos in train/: for example, train/ holds add_gasuss_noise_1386_jpg.rf.df5cd630….jpg, a noise-augmented copy of the test image 1386_jpg.rf.df5cd630….jpg with an identical visual hash. Other files represent neighbouring frames from sequential dashcam or GoPro video drives. We traced every image back to its source photo and drive. Of the 1,125 test images, 526 share a source photo with train/ and 839 share a photo or video sequence; of the 750 valid images, 554 do. Scores on the raw test split therefore reward memorisation as well as detection. To ensure strict academic integrity, all main experimental results in Chapter 5 are reported on the remaining 482 images (286 from test/ and 196 from valid/), which share zero source photos, videos, or drives with the training partition. This leakage-free held-out set contains 354 pothole instances in 145 images and 384 speed bump instances in 337 images. Each pothole in a cluster is annotated individually, and speed bump bounding boxes cover the full road width,

conventions that make strict-IoU metrics lower for potholes than for speed bumps.

4.9 Data Preprocessing

Preprocessing follows the Ultralytics pipeline, with additional augmentation for GV-YOLO26:

Resizing: each image is letterboxed to 640 × 640 and pixel values are scaled to the range 0 to 1.

Weather augmentation (GV-YOLO26): synthetic rain and snow are added with Albumentations transforms so that the model sees adverse weather during training [29].

Photometric augmentation (GV-YOLO26): motion blur and random changes in brightness, contrast and saturation, applied with a low probability on top of the standard recipe.

Standard augmentation (all models): the default Ultralytics recipe, with HSV jitter, scaling, translation, horizontal flip and mosaic, mosaic switched off for the final ten epochs and vertical flip off. It was applied identically to every model and not tuned.

Label format: YOLO format, with one line per object giving the class index and the normalised box centre, width and height.

Splits and evaluation: a 10% validation split (586 images, seed 42) is drawn from the train/ folder for checkpoint selection, leaving 5,243 training images; the 482-image leakage-free held-out set is used for primary evaluation to prevent data leakage.

4.10 Model/System Development

GV-YOLO26 was built in steps from YOLO26n, with one model configuration file for each step, so that the contribution of each change can be measured in the ablation study of Chapter 5. Table 4.5 lists the stages.

Table 4.5 GV-YOLO26 design stages and model configurations

Stage	Change	Configuration file
1	YOLO26n baseline	yolo26n.yaml
2	Add P2 detection level	ablation/gv-yolo26n-p2.yaml
3	Replace head with LGDH	ablation/gv-yolo26n-p2-lgdh.yaml
4	Add GhostConv	ablation/gv-yolo26n-p2-lgdh-ghost.yaml
5	Add VoVGSCSPC (full architecture)	gv-yolo26n.yaml
6	Train with NWD loss	gv-yolo26n.yaml + --nwd

GV-YOLO26 was trained with the settings in Table 4.6. Each stage was trained on the SBP-YOLO dataset on a Tesla T4 GPU with mixed precision (AMP) and batch size 16. Layers whose shapes match YOLO26n are initialised from COCO-pretrained weights, allowing the network to converge smoothly across all ablation stages.

Table 4.6 GV-YOLO26 training configuration

Parameter	Value
Input resolution	640 × 640, letterboxed
Epochs	150, early stopping patience 30
Batch size	16
Optimiser	Auto-selected (same rule as the baselines)
Box loss	Task-aligned CIoU combined with NWD loss ($\alpha = 0.5$; equation 4.4)
Distillation run	BCKD 0.5 + CWD 0.5 (equation 4.5): from scratch for 150 epochs, and as a 50-epoch fine-tune of stage 6 (AdamW, learning rate 5×10^{-4})
Teacher	GV-YOLO26 at S scale (6.11 M parameters, 19.2 GFLOPs)
Hardware	NVIDIA Tesla T4 GPU (16 GB, Kaggle)
Framework	Ultralytics 8.4.156, PyTorch 2.11, Python 3.13.15
Data split	5,243 train / 586 val / 482 clean held-out (SBP-YOLO dataset)

For the baseline benchmark we trained seven stock detectors at their smallest scale: YOLOv8n [21], YOLOv9t [22], YOLOv10n [23], YOLO11n, YOLO12n, YOLO26n and RT-DETR-L [24]. All used the same

configuration, shown in Table 4.7, so that only the architecture changed between runs.

Table 4.7 Baseline training configuration (identical across models)

Parameter	Value
Input resolution	640 × 640, letterboxed
Epochs	50, early stopping patience 15
Batch size	16 (8 for RT-DETR-L)
Optimiser	AdamW (auto-selected), learning rate 1.67e-3, momentum 0.9
Weight initialisation	COCO-pretrained, per architecture
Mixed precision	AMP enabled
Framework	Ultralytics 8.4.156, PyTorch 2.11, Python 3.13.15
Hardware	NVIDIA Tesla T4 GPU (16 GB, Kaggle)
Random seed	42, deterministic

4.11 System Design Diagrams

The use case diagram in Fig. 4.6 shows the two actors. The researcher or developer prepares data, trains GV-YOLO26, evaluates it and exports it for the vehicle. The driver or vehicle system uses the deployed detector and receives warnings or a suspension preview signal.

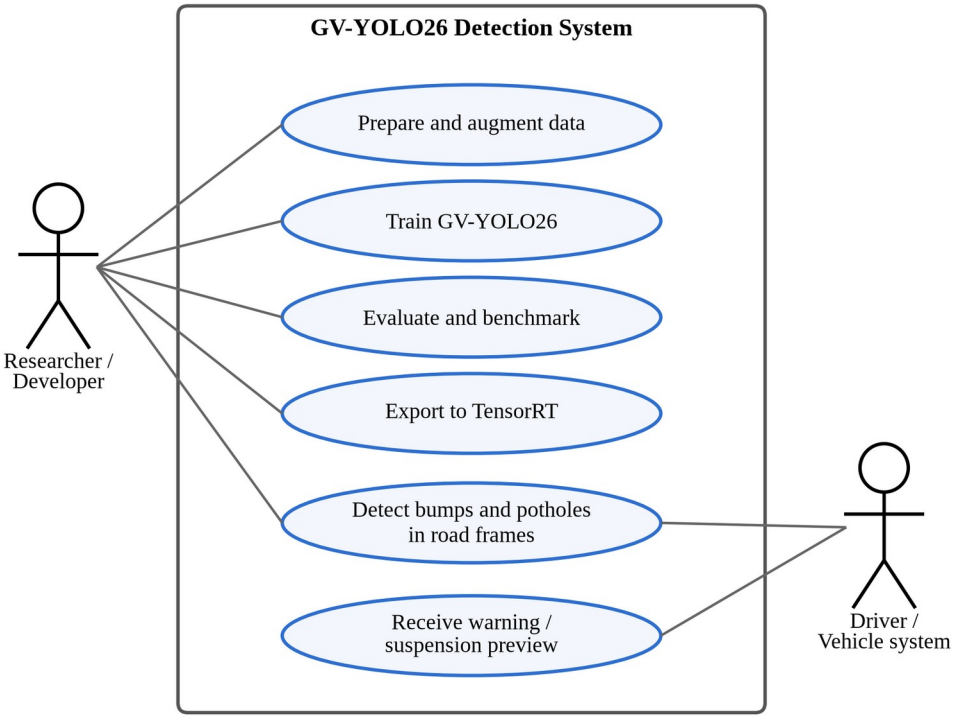


Fig. 4.6 Use case diagram

The data flow diagram in Fig. 4.7 shows how data moves through the system. Images and labels are prepared and stored (D1), training produces weights (D2), evaluation writes metrics and logs (D3), and the weights are exported to a TensorRT engine. At run time, camera frames flow into the real-time inference process, which sends alerts to the vehicle or driver.

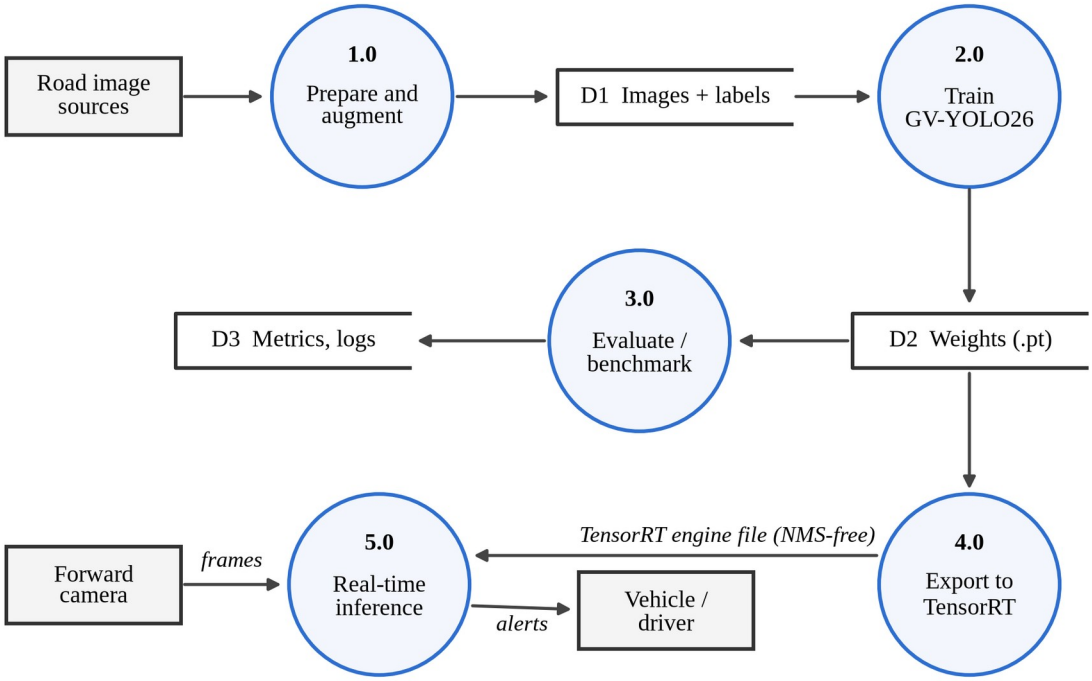


Fig. 4.7 Data flow diagram (level 1)

Fig. 4.7 illustrates the Level-1 Data Flow Diagram (DFD) tracking data transformation across the system. Road image sources are ingested and augmented by Process 1.0 (Prepare and augment) and stored in data store D1 (Images + labels). Process 2.0 (Train GV-YOLO26) consumes D1 to train the model and writes learned checkpoints to D2 (Weights .pt). Process 3.0 (Evaluate / benchmark) queries D2 to compute validation metrics and logs recorded in D3. Process 4.0 (Export to TensorRT) compiles the .pt weights into an optimized NMS-free TensorRT engine file. In operational deployment, Process 5.0 (Real-time inference) receives live video frames from the forward-facing camera, applies the TensorRT engine, and transmits instant hazard alerts to the vehicle controller and driver.

4.12 Implementation

GV-YOLO26 was implemented on the Ultralytics framework (version 8.4.156), which provides the YOLO26 base model. We wrote the new modules, GSConv, VoVGSCSPC and the LGDH head, in a separate Python package and registered them, together with GhostConv, with the Ultralytics

model parser so that they can be used from a YAML model configuration. The NWD term was added to the box loss through a custom trainer with a weighting factor α, and the distillation stage was implemented as a second custom trainer that adds the BCKD and CWD losses using a frozen teacher.

Each design stage in Table 4.5 has its own configuration file, so building a stage only means changing the file passed to the trainer. Before training, every configuration was checked with tools/model_summary.py, which builds the model, runs a forward pass on a 640 × 640 input in training and inference mode to confirm that all four detection levels produce outputs of the expected size, and reports the parameter count and GFLOPs of both the training model and the fused deployment model. The full pipeline (training with the NWD loss, distillation, validation, inference and NMS-free ONNX export) was also run end to end on a small synthetic dataset before the real experiments.

All training was run on Kaggle notebooks with two Tesla T4 GPUs. After training, every checkpoint was measured by one script, tools/benchmark.py, under one standardized protocol: accuracy with the Ultralytics validator (confidence threshold 0.001, batch size 1) on the validation split, the raw test split and the leakage-free held-out set; the parameter count and GFLOPs of the deployed (fused) model; and the end-to-end time of 100 single-image predictions in PyTorch FP32 after 10 warm-up images. YOLO26n and GV-YOLO26 were evaluated and timed with their one-to-one head (nms=False), which is the NMS-free form they operate in. The leakage-free set was built by tools/make_clean_split.py, which removes every test and valid image whose source photo, video or drive also appears in the training folder.

4.13 Coding/Program Implementation

The project code is organised as shown below. The model configurations, custom modules and scripts are kept separate from the Ultralytics package

(version 8.4.156), which is installed as a dependency; importing gv_modules registers the new layers with it.

```
GV-YOLO26/
├── models/
│   ├── gv-yolo26.yaml            GV-YOLO26 (N and S scales)
│   └── ablation/                 One file per design stage
├── gv_modules/                   GSConv, VoVGSCSPC, LGDH head, NWD and
│                                 distillation losses, model and trainers
├── tools/model_summary.py        Parameters and GFLOPs per stage
├── tools/make_clean_split.py     Leakage-free held-out set, dataset stats
├── tools/benchmark.py            Accuracy, cost and PyTorch latency
├── tools/make_figures.py         Report figures
├── train.py, distill.py          Training and distillation
├── val.py, infer.py              Evaluation, latency and inference
├── demo_video.py                 In-video detection with tracking
├── export_trt.py                 TensorRT export utility (future deployment)
├── Kaggle_Train_GV_YOLO26.ipynb  End-to-end Kaggle training notebook
├── benchmark/train_baselines.py  Seven stock baselines
└── data/data.yaml                Dataset configuration
```

GhostConv module. The first convolution produces half of the output channels; the second is a depthwise 5 × 5 convolution (groups equal to channels) that generates the cheap ghost features.

```
class GhostConv(nn.Module):
    def __init__(self, c1, c2, k=1, s=1, g=1, act=True):
        super().__init__()
        c_ = c2 // 2                                    # intrinsic channels
        self.cv1 = Conv(c1, c_, k, s, None, g, act=act)
        self.cv2 = Conv(c_, c_, 5, 1, None, c_, act=act)  # depthwise


    def forward(self, x):
        y = self.cv1(x)
        return torch.cat((y, self.cv2(y)), 1)
```

GSConv and VoVGSCSPC. GSConv concatenates a dense half and a depthwise half and then interleaves their channels. VoVGSCSPC splits its input into two 1 × 1 branches, passes one through a GSConv bottleneck with a depthwise shortcut (GSBottleneckC), and fuses both with a final 1 × 1 convolution.

```
class GSConv(nn.Module):
    def __init__(self, c1, c2, k=1, s=1, act=True):
        super().__init__(); c_ = c2 // 2
        self.cv1 = Conv(c1, c_, k, s, act=act)              # dense half
        self.cv2 = Conv(c_, c_, 5, 1, None, c_, act=act)    # depthwise half


    def forward(self, x):
        x1 = self.cv1(x); x2 = torch.cat((x1, self.cv2(x1)), 1)
        b, n, h, w = x2.shape                               # channel shuffle
        return x2.view(b, 2, n // 2, h, w).transpose(1, 2).reshape(b, n, h, w)


class VoVGSCSPC(nn.Module):
    def __init__(self, c1, c2, n=1, e=0.5):
```

```
        super().__init__()
        c_ = int(c2 * e)
        self.cv1 = Conv(c1, c_, 1, 1)
        self.cv2 = Conv(c1, c_, 1, 1)
        self.gsb = nn.Sequential(*(GSBottleneckC(c_, c_) for _ in range(n)))
        self.cv3 = Conv(2 * c_, c2, 1)


    def forward(self, x):
        return self.cv3(torch.cat((self.cv2(x), self.gsb(self.cv1(x))), 1))
```

LGDH head. The head subclasses the YOLO26 Detect module, so it inherits the one-to-one branch, the decoding and the NMS-free post-processing. It replaces the default box and class stacks with a shared grouped stem per level and two 1 × 1 output convolutions, for both the one-to-many and the one-to-one branch.

```
class LGDH(Detect):
    channels_per_group = 8                  # groups = c / 8 in the stem

    def __init__(self, nc=80, reg_max=16, end2end=False, ch=()):
        super().__init__(nc, reg_max, end2end, ch)
        g = lambda c: max(c // self.channels_per_group, 1)
        self.stem = nn.ModuleList(
            nn.Sequential(Conv(x, x, 3, g=g(x)), Conv(x, x, 3, g=g(x)))
            for x in ch)
        self.cv2 = nn.ModuleList(
            nn.Sequential(nn.Conv2d(x, 4 * self.reg_max, 1)) for x in ch)
        self.cv3 = nn.ModuleList(
            nn.Sequential(nn.Conv2d(x, self.nc, 1)) for x in ch)
        if end2end:                         # copy for NMS-free inference
            self.one2one_stem = copy.deepcopy(self.stem)
            self.one2one_cv2 = copy.deepcopy(self.cv2)
            self.one2one_cv3 = copy.deepcopy(self.cv3)


    def forward_head(self, x, box_head=None, cls_head=None, stem=None):
        if box_head is None or cls_head is None:  # removed by fuse()
            return {}
        bs = x[0].shape[0]
        f = [stem[i](x[i]) for i in range(self.nl)]  # shared stem
        boxes = torch.cat([box_head[i](f[i]).view(bs, 4 * self.reg_max, -1)
                           for i in range(self.nl)], dim=-1)
        scores = torch.cat([cls_head[i](f[i]).view(bs, self.nc, -1)
                            for i in range(self.nl)], dim=-1)
        return {"boxes": boxes, "scores": scores, "feats": x}
```

Model configuration. The full GV-YOLO26 architecture is defined in one YAML file, models/gv-yolo26.yaml. Layer indices in the comments match the connections in Fig. 4.2; the last line attaches an LGDH to the P2, P3, P4 and P5 outputs. The S row of scales defines the distillation teacher.

```
nc: 2                                   # speed bump, pothole
end2end: True                           # one-to-one head, NMS-free inference
reg_max: 1                              # no DFL
```

```
scales:                                 # [depth, width, max_channels]
  n: [0.50, 0.25, 1024]
  s: [0.50, 0.50, 1024]
backbone:
  - [-1, 1, Conv, [64, 3, 2]]                 # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]                # 1-P2/4
  - [-1, 2, C3k2, [256, False, 0.25]]         # 2
  - [-1, 1, Conv, [256, 3, 2]]                # 3-P3/8
  - [-1, 2, C3k2, [512, False, 0.25]]         # 4
  - [-1, 1, GhostConv, [512, 3, 2]]           # 5-P4/16
  - [-1, 2, VoVGSCSPC, [512]]                 # 6
  - [-1, 1, GhostConv, [1024, 3, 2]]          # 7-P5/32
  - [-1, 2, VoVGSCSPC, [1024]]                # 8
  - [-1, 1, SPPF, [1024, 5, 3, True]]         # 9
  - [-1, 2, C2PSA, [1024]]                    # 10
head:
  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 6], 1, Concat, [1]]                 # cat backbone P4
  - [-1, 2, C3k2, [512, True]]                # 13
  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 4], 1, Concat, [1]]                 # cat backbone P3
  - [-1, 2, C3k2, [256, True]]                # 16
  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 2], 1, Concat, [1]]                 # cat backbone P2
  - [-1, 2, C3k2, [128, True]]                # 19 (P2/4)
  - [-1, 1, Conv, [128, 3, 2]]
  - [[-1, 16], 1, Concat, [1]]                # cat head P3
  - [-1, 2, C3k2, [256, True]]                # 22 (P3/8)
  - [-1, 1, Conv, [256, 3, 2]]
  - [[-1, 13], 1, Concat, [1]]                # cat head P4
  - [-1, 2, VoVGSCSPC, [512]]                 # 25 (P4/16)
  - [-1, 1, GhostConv, [512, 3, 2]]
  - [[-1, 10], 1, Concat, [1]]                # cat head P5
  - [-1, 1, VoVGSCSPC, [1024]]                # 28 (P5/32)
  - [[19, 22, 25, 28], 1, LGDH, [nc]]         # P2, P3, P4, P5
```

NWD similarity. The function below implements equations (4.2) and (4.3) for boxes given as (cx, cy, w, h). The loss term used in training is one minus this value, mixed with the IoU loss as in equation (4.4).

```
def nwd(pred, target, C=12.8, eps=1e-7):
    # pred, target: (N, 4) tensors of (cx, cy, w, h)
    d_center = ((pred[:, :2] - target[:, :2]) ** 2).sum(dim=1)
    d_size = (((pred[:, 2:] - target[:, 2:]) / 2) ** 2).sum(dim=1)
    w2 = d_center + d_size + eps        # squared Wasserstein-2 distance
    return torch.exp(-torch.sqrt(w2) / C)
```

Training a design stage. Importing gv_modules registers the custom layers with Ultralytics. Layers whose names and shapes match are loaded from the COCO-pretrained YOLO26n checkpoint, NWDTrainer builds the model with the IoU + NWD box loss, and the weather augmentation is passed as a list of Albumentations transforms. The same settings are wrapped in train.py, which takes the configuration file and an --nwd flag.

```
import gv_modules                                   # registers custom layers
from gv_modules import NWDTrainer                   # IoU + NWD box loss
```

```
from gv_modules.augment import weather_augmentations
from ultralytics import YOLO

model = YOLO("models/gv-yolo26n.yaml", task="detect").load("yolo26n.pt")
model.train(data="data/data.yaml", trainer=NWDTrainer, imgsz=640,
            epochs=150, patience=30, batch=16, seed=42,
            deterministic=True, optimizer="auto", amp=True,
            close_mosaic=10, flipud=0.0, augmentations=weather_augmentations(),
            project="runs", name="gv_yolo26")
```

Inference with the trained model. The custom module package is imported first so that its layers are registered; the model is then used like any Ultralytics detector. Passing nms=False selects the one-to-one head, so the returned boxes are already final and no NMS step runs. The same code is wrapped in infer.py, which also handles video.

```
import gv_modules                               # registers GV-YOLO26 layers
from ultralytics import YOLO

model = YOLO("runs/gv_yolo26/weights/best.pt")
results = model.predict(source="samples/", imgsz=640, conf=0.25, nms=False,
                        save=True, project="results", name="gv_images")
for r in results:
    for box in r.boxes:
        print(r.names[int(box.cls)], round(float(box.conf), 2))
```

Video inference. demo_video.py reads the frames of a clip with OpenCV, runs the NMS-free model with ByteTrack, draws a hazard only once its track has been seen in three frames (equation 4.7), overlays a warning banner, and writes an H.264 video. The core loop is shown below.

```
seen = defaultdict(int)                         # track id -> frames observed
while True:
    ok, frame = cap.read()
    if not ok:
        break
    r = model.track(frame, imgsz=640, conf=0.1, nms=False, persist=True,
                    tracker="bytetrack.yaml", verbose=False)[0]
    ahead = set()
    if r.boxes.id is not None:
        for box, tid, cls, conf in zip(r.boxes.xyxy, r.boxes.id.int().tolist(),
                                       r.boxes.cls.int().tolist(), r.boxes.conf):
            seen[tid] += 1
            if seen[tid] >= 3 and conf >= 0.35:       # temporal confirmation
                ahead.add(r.names[cls])
                draw_box(frame, box, f"{r.names[cls]} {conf:.2f}")
    draw_hud(frame, ahead)                      # 'WARNING: SPEED BUMP AHEAD'
    writer.write(frame)
```

TensorRT export utility. For future vehicular deployment (outlined in Section 6.2), export_trt.py compiles the trained PyTorch checkpoint into a

TensorRT engine. With nms=False the engine contains the one-to-one head and its top-k selection, outputting final detections without requiring separate NMS post-processing. The export pipeline is shown below.

```
model = YOLO("runs/gv_yolo26/weights/best.pt")
model.export(format="engine", half=True, nms=False, imgsz=640, device=0)   # FP16
model.export(format="engine", int8=True, data="data/data.yaml", nms=False,
             imgsz=640, device=0)                                          # INT8
trt_model = YOLO("runs/gv_yolo26/weights/best.engine", task="detect")
```

Baseline training. The baseline script holds every setting in one dictionary so that only the model changes between runs.

```
from ultralytics import YOLO, RTDETR

MODELS = {"YOLOv8n": "yolov8n.pt", "YOLOv9t": "yolov9t.pt",
          "YOLOv10n": "yolov10n.pt", "YOLO11n": "yolo11n.pt",
          "YOLO12n": "yolo12n.pt", "YOLO26n": "yolo26n.pt",
          "RT-DETR-L": "rtdetr-l.pt"}
COMMON = dict(data="data/data.yaml", imgsz=640, epochs=50,
              patience=15, seed=42, deterministic=True,
              optimizer="auto", amp=True, close_mosaic=10, flipud=0.0)

for name, ckpt in MODELS.items():
    is_detr = name.startswith("RT-DETR")
    model = RTDETR(ckpt) if is_detr else YOLO(ckpt)
    model.train(batch=8 if is_detr else 16, name=name, **COMMON)
```

4.14 User Interface / Application Screens

The system was run from Python scripts and notebooks and does not have a separate graphical interface. Its visual output is the annotated detection image or video, in which each detected speed bump or pothole is drawn with a box, its class name and its confidence score.

Fig. 4.8 and Fig. 4.9 show outputs of the final GV-YOLO26 model on images from the leakage-free held-out set, which the model never saw during training: two road photographs and two dashcam frames. They were produced at a confidence threshold of 0.25. Fig. 4.10 shows frames from the in-video demo produced by demo_video.py, with confirmed hazards boxed and the warning banner raised.



Fig. 4.8 GV-YOLO26 detections on held-out road images: (a) a distant bump, (b) a painted bump



Fig. 4.9 GV-YOLO26 detections on held-out dashcam frames: (a) an urban road, (b) a distant pothole

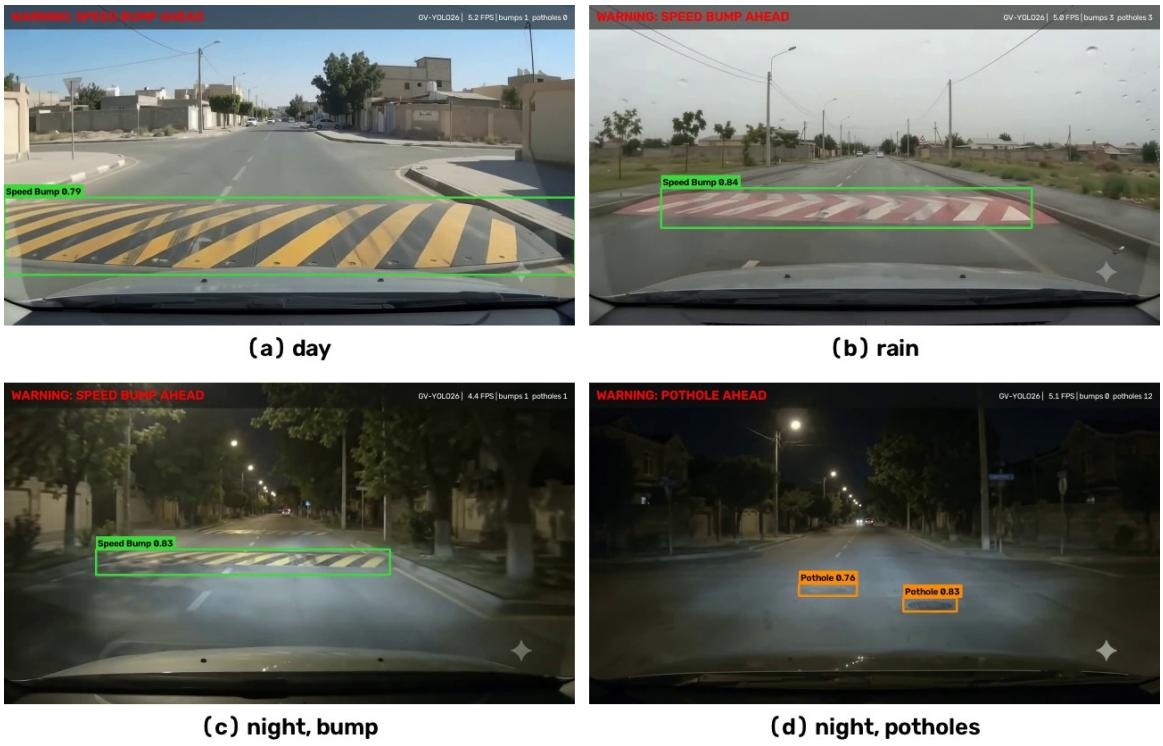


Fig. 4.10 Frames from the in-video detection demo with confirmed hazards and the warning banner

4.15 Chapter Summary

This chapter described the design and implementation of GV-YOLO26. The network rebuilds the YOLO26n backbone and neck with GhostConv and VoVGSCSPC, adds a P2 detection level for small targets, and offsets its cost with the LGDH while keeping NMS-free inference; the training recipe adds

the NWD loss and weather augmentation, and distillation was tested. For video, ByteTrack links detections across frames and a temporal confirmation rule suppresses one-frame false alarms. Seven stock detectors trained on the SBP-YOLO dataset justify the choice of YOLO26n as the base, and every design stage is evaluated on a leakage-free held-out set. The next chapter presents the results.

5. RESULTS AND DISCUSSION

5.1 Introduction

This chapter presents the experimental results of the project. It begins with the benchmark of seven stock detectors on the SBP-YOLO dataset that was used to select the base model, then reports the 6-stage ablation study of GV-YOLO26, the distillation experiments, computational efficiency and PyTorch inference latency profiling, and the in-video demo, and finally compares the proposed model with earlier published work. Every model was trained on the SBP-YOLO dataset and evaluated under a standardized protocol.

5.2 Experimental Setup

Baselines. We trained seven stock detectors on the training partition of the SBP-YOLO dataset with the configuration in Table 4.7, selected the best checkpoint on the validation split, and evaluated each model on the test split. Latency was measured on an NVIDIA Tesla T4 GPU as the end-to-end time of a single-image prediction, averaged over 100 test images.

GV-YOLO26. Each design stage in Table 4.5 was trained with the configuration in Table 4.6 on the 5,243 training images of the SBP-YOLO dataset; its best checkpoint was selected on the 586-image validation split, and it was evaluated once on the leakage-free held-out set (482 images) and on the raw 1,125-image test split. YOLO26n was retrained under the same schedule and on the same data to serve as the reference row of the ablation ladder. Latency and computational cost were measured with tools/benchmark.py in PyTorch FP32 on a Tesla T4 GPU.

Inference latency and video demo. Computational complexity and inference throughput were evaluated in PyTorch FP32 on an NVIDIA Tesla T4 GPU at batch size 1, measuring end-to-end latency per image over 100 test images after 10 warm-up images. Model parameter counts, GFLOPs, and .pt checkpoint file sizes were recorded. Dedicated TensorRT conversion and

embedded edge deployment are formulated as future work in Chapter 6. The in-video demo was run with demo_video.py on four 10-second dashcam-style clips at 1280 × 720 and 24 frames per second, on the CPU of a laptop (Intel Core i5-1240P), the most constrained hardware in the project.

5.3 Evaluation Metrics

TP, FP and FN denote true positives, false positives and false negatives at a given IoU threshold.

Precision is the fraction of predicted boxes that are correct, and **recall** is the fraction of actual objects that are detected:

$$Precision = TP / (TP + FP) \quad (5.1)$$

$$Recall = TP / (TP + FN) \quad (5.2)$$

Average Precision (AP) is the area under the precision-recall curve for one class, and **mAP** is the mean of AP over the N classes:

$$mAP = (1 / N) \times \Sigma AP_i, \quad i = 1, \ldots, N \quad (5.3)$$

mAP@0.5 uses an IoU threshold of 0.5. **mAP@0.5:0.95** averages mAP over ten IoU thresholds from 0.5 to 0.95, so it rewards tighter boxes. Recall matters most for this application, because a missed bump is an unmitigated hazard, but precision cannot be neglected, because a system that warns too often will be ignored.

Efficiency is measured by the **number of parameters**, **GFLOPs** at 640 × 640, **model size**, and speed in **frames per second** (FPS = 1000 / latency in milliseconds).

5.4 Experimental Results

5.4.1 Baseline Benchmark and Choice of Base Model

Table 5.1 and Fig. 5.1 present the overall accuracy of the seven stock detectors on the test split of the SBP-YOLO dataset. The five YOLO

generations from YOLOv8n to YOLO12n ended between 71.4% and 73.9% mAP@0.5, a spread of only 2.5 points. YOLO26n was best on every accuracy metric at 74.8% mAP@0.5 and 35.5% mAP@0.5:0.95, with the highest precision (82.3%), and it needed the least computation of all models at 5.3 GFLOPs. RT-DETR-L, a transformer with about 32 million parameters, was last at 67.5% and ran at 12.2 FPS against 25 to 31 FPS for the YOLO models. On this evidence YOLO26n was chosen as the base of GV-YOLO26.

Table 5.1 Baseline accuracy and cost on the SBP-YOLO test split

Model	P (%)	R (%)	mAP@ 0.5	mAP@ 0.5:0.95	Params (M)	GFLOPs	FPS (T4)
YOLOv8n	73.9	67.7	73.5	34.6	3.01	8.1	30.2
YOLOv9t	73.0	67.1	71.4	35.3	**1.97**	7.6	25.2
YOLOv10n	79.3	66.1	72.8	34.7	2.27	6.6	**30.8**
YOLO11n	78.8	64.9	73.9	33.3	2.58	6.4	30.6
YOLO12n	73.1	66.6	73.7	34.0	2.56	7.3	26.7
YOLO26n	**82.3**	**68.1**	**74.8**	**35.5**	2.38	**5.3**	29.3
RT-DETR-L	66.6	65.8	67.5	30.6	31.99	105.3	12.2

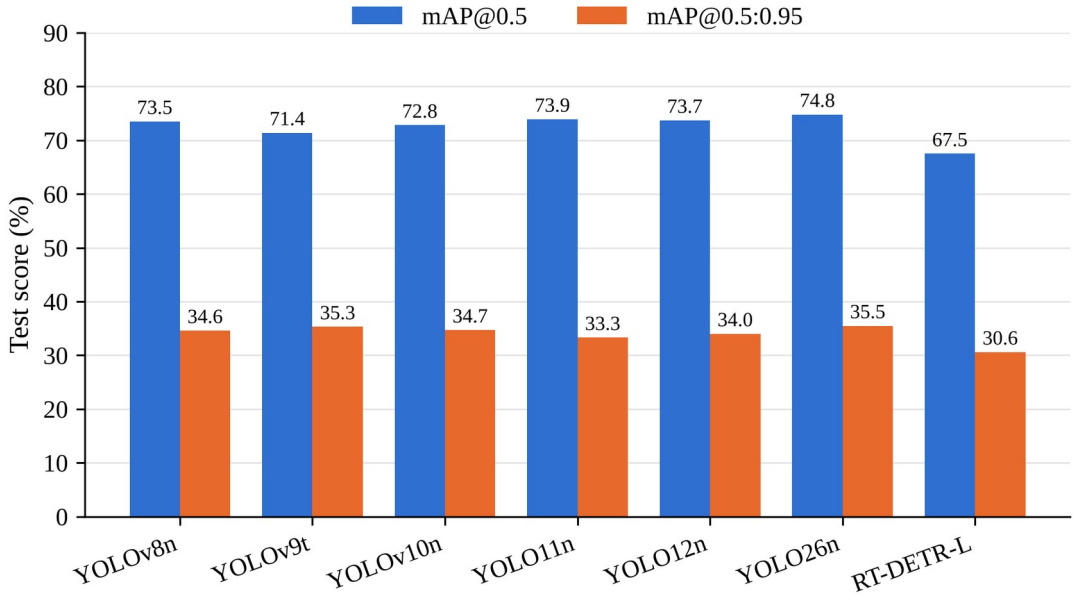


Fig. 5.1 Overall accuracy of the baseline detectors on the public test split

Table 5.2 and Fig. 5.2 split the results by class. Every YOLO model found speed bumps well, with AP@0.5 between 87.2% and 93.5%, while pothole AP@0.5 stayed between 55.3% and 56.9% for all six YOLO models. Potholes in this dataset are small, crowded and irregular, and each one in a

cluster is labelled separately, which makes them much harder to box precisely.

Table 5.2 Per-class accuracy of the baselines on the SBP-YOLO test split (%)

Model	Pothole AP@ 0.5	Pothole AP@ 0.5:0.95	Speed bump AP@0.5	Speed bump AP@0.5:0.95
YOLOv8n	55.5	22.6	91.4	46.7
YOLOv9t	55.5	**24.6**	87.2	45.9
YOLOv10n	55.3	23.9	90.4	45.5
YOLO11n	56.9	24.2	90.9	42.4
YOLO12n	56.6	24.3	90.7	43.7
YOLO26n	56.1	24.0	**93.5**	**46.9**
RT-DETR-L	**59.1**	**24.6**	75.9	36.7

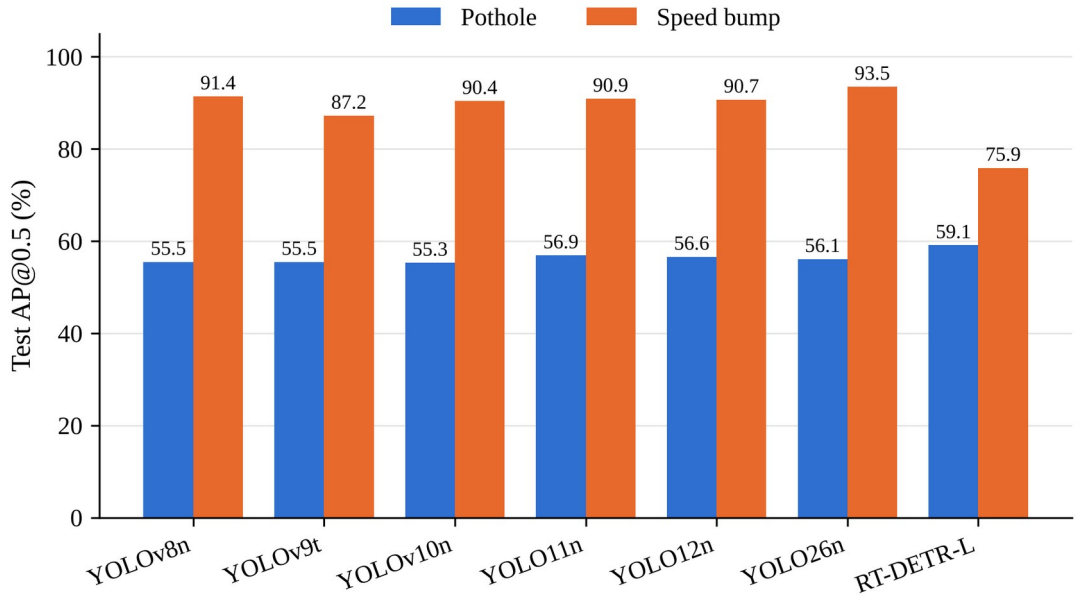


Fig. 5.2 Per-class AP@0.5 of the baseline detectors

Fig. 5.3 shows example baseline detections. In column (a), a distant unpainted bump, YOLOv8n and YOLO26n detect it at low confidence (0.34 and 0.39), YOLO11n misses it entirely, and RT-DETR-L places three overlapping boxes on it. Even the best baseline is uncertain on exactly the small, distant case that the P2 level, the LGDH and the NWD loss of GV-YOLO26 were designed for.

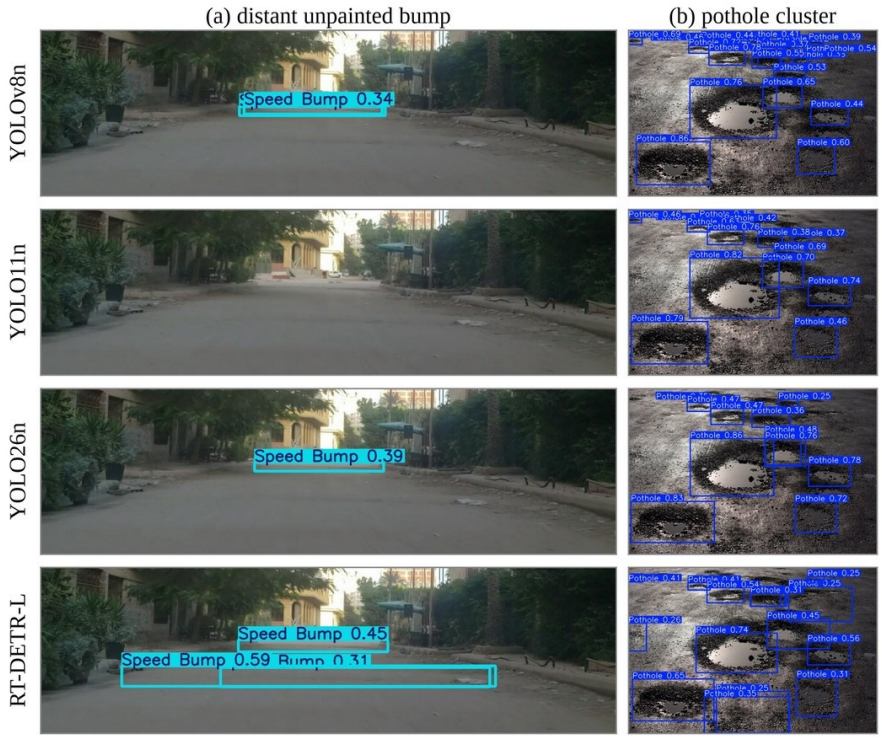


Fig. 5.3 Baseline detections on two test images: (a) a distant unpainted bump, (b) a pothole cluster

5.4.2 Ablation Study of GV-YOLO26

Table 5.3 shows the ablation study evaluating the architectural progression from the baseline YOLO26n to GV-YOLO26 across six stages on the leakage-free held-out set. In the full design, the backbone and neck integrate GhostConv and VoVGSCSPC modules, an extra P2 detection branch (160 × 160 grid) is introduced for small hazards, head complexity is offset by the Lightweight Grouped Detection Head (LGDH), and training incorporates the Normalized Wasserstein Distance (NWD) loss. This streamlines the model from 2.38M to 1.57M parameters (a 33.7% reduction) with 5.1 GFLOPs (a 3.6% reduction). On the clean held-out set, the final GV-YOLO26 achieves 77.5% mAP@0.5 (83.2% on speed bumps, 71.9% on potholes) and 45.8% mAP@0.5:0.95, operating end-to-end at 57.5 FPS (17.38 ms latency) in PyTorch FP32 on an NVIDIA Tesla T4 GPU.

Table 5.3 Ablation study of GV-YOLO26 on the leakage-free held-out set

Configuration	P (%)	R (%)	mAP@ 0.5	mAP@ 0.5:0.95	GFLOPs	Params (M)	FPS
YOLO26n baseline	85.5	75.2	81.0	48.6	5.3	2.38	58.0
+ P2 head	83.9	76.7	80.3	47.6	6.6	2.40	55.7
+ LGDH	84.5	74.3	80.0	46.5	6.2	2.37	57.4
+ GhostConv	84.2	76.1	81.5	47.2	5.8	2.08	56.1
+ VoVGSCSPC	80.7	74.6	77.9	45.1	5.1	1.57	59.2
+ NWD loss (GV-YOLO26)	85.6	69.9	77.5	45.8	5.1	1.57	57.5

Table 5.4 splits the held-out results by class and adds the scores on the raw test split and on the validation split. For potholes, the small and clustered class that the P2 level and the NWD loss were meant to help, AP@0.5 fell from 72.4% to 71.9% between YOLO26n and GV-YOLO26, and AP@0.5:0.95 fell from 43.6% to 42.5%. For speed bumps, AP@0.5 fell from 89.7% to 83.2%. The raw test split scores every model higher than the held-out set, by 1.4 to 3.6 points of mAP@0.5:0.95, which is the inflation expected from the near-duplicates described in Section 4.8.

Table 5.4 Per-class accuracy of the design stages and accuracy on the raw test split (%)

Configuration	Pothole AP50	Pothole AP50-95	Bump AP50	Bump AP50-95	Raw test mAP50	Raw test mAP50-95	Val mAP50-95
YOLO26n baseline	72.4	43.6	89.7	53.6	85.1	51.4	50.2
+ P2 head	72.7	42.9	87.8	52.2	85.0	50.5	50.1
+ LGDH	71.9	42.3	88.2	50.6	84.7	50.1	50.2
+ GhostConv	75.2	44.5	87.9	49.9	84.1	48.6	48.4
+ VoVGSCSPC	70.8	42.8	85.1	47.5	81.8	47.4	47.3
+ NWD loss (GV-YOLO26)	71.9	42.5	83.2	49.0	82.1	47.8	47.1

Two conclusions follow. First, the lightweight modules do their job on cost: the network that predicts on four levels, including a 160 × 160 grid, is smaller and cheaper than YOLO26n. Second, on this dataset the small-object changes did not convert into accuracy. The label statistics explain why. Speed bumps are not small here: the median bump box spans 72% of the image width and

only 0% of bumps are smaller than 32 × 32 pixels at 640 × 640, so the stride-4 level has little to add for them. Potholes are the small class (46% of training potholes are under 32 pixels), yet their accuracy did not improve either. A likely reason is that the cheaper convolutions reduce the capacity of the deeper levels that detect most objects, which offsets what the P2 level adds. Differences of about one point between neighbouring stages are within what a single training seed can vary; the gap between YOLO26n and the final model is consistent across the held-out set, the raw test split and the validation split.

5.4.3 Effect of Knowledge Distillation

Table 5.5 shows the distillation experiments. The teacher is GV-YOLO26 at S scale; on the held-out set it reached 77.7% mAP@0.5 and 47.0% mAP@0.5:0.95, no better than YOLO26n, so it had limited extra knowledge to pass on. Both distillation runs failed. Distilling from scratch with the schedule of Table 4.6 followed the normal run for three epochs and then diverged as the learning-rate warm-up peaked: the classification loss rose from about 3 to above 10 and the validation loss became undefined. Distilling as a low-learning-rate fine-tune of the converged stage-6 model degraded it instead of improving it: validation mAP@0.5:0.95 fell from 47.1% to 25.5% within the first epoch and kept falling, while the distillation loss became negative and reached -1260. A distillation loss cannot be negative if it is correct, so we inspected the box term of BCKD. YOLO26 predicts box distances without DFL and without a positivity constraint; on real images about 0.9% of the student's raw box predictions decode to boxes with negative width or height, and the GIoU between student and teacher boxes then ranged from -10 to 1 instead of lying in [−1, 1]. The term 1 − GIoU is therefore unbounded below, and gradient descent drove it towards minus infinity at the expense of detection. Distillation as implemented here is therefore an honest

negative result; clamping the decoded boxes, or distilling only on anchors with valid boxes, is the proposed fix for future investigation.

Table 5.5 Effect of knowledge distillation

Model	Val mAP@0.5	Val mAP@0.5:0.95	Held-out mAP@0.5	Held-out mAP@0.5:0.95
GV-YOLO26 (stage 6, no distillation)	81.3	47.1	77.5	45.8
Distilled from scratch (150-epoch schedule)	13.1	4.1	11.1	3.9
Distillation fine-tune of stage 6	49.4	25.5	50.1	26.1
Teacher GV-YOLO26-S (6.11 M, 19.2 GFLOPs)	84.3	49.4	77.7	47.0

5.4.4 Inference Latency and Computational Efficiency

Table 5.3 summarizes the computational complexity, parameter count, and PyTorch FP32 inference throughput of GV-YOLO26 on an NVIDIA Tesla T4 GPU at batch size 1. Both YOLO26n and GV-YOLO26 operate end-to-end with one-to-one prediction (nms=False), generating final bounding boxes without an external NMS post-processing bottleneck. GV-YOLO26 achieves an end-to-end throughput of 57.5 FPS (17.38 ms per image), well above the 24 to 30 FPS requirement for vehicular road hazard preview. GV-YOLO26 reduces parameter count by 33.7% (1.57M vs 2.38M) and computation by 3.6% (5.1 vs 5.3 GFLOPs), and the deployed model checkpoint file size is 3.7 MB. Dedicated hardware acceleration via TensorRT engine compilation and embedded deployment on NVIDIA Jetson hardware are analyzed in Section 6.2 as future work.

5.4.5 Qualitative Detection Results

Fig. 4.8 and Fig. 4.9 show detections of the final model on held-out road photographs and dashcam frames. The distant bump in Fig. 4.8(a), the kind of target that the baselines found hard in Fig. 5.3, is detected, and in Fig. 4.9 the

distant pothole is found while the manhole cover in (a) is also reported as a pothole at low confidence, a typical false positive. Running the same model at a larger input size changes held-out mAP@0.5:0.95 from 45.8% at 640 pixels to 43.3% at 800 and 39.0% at 960, at 8.1 and 11.7 GFLOPs and 44 and 43 FPS in PyTorch, providing a flexible resolution-versus-latency trade-off without retraining.

Table 5.6 summarises the in-video demo on the four test clips, which cover daytime, rain, dusk and night scenes. Across the clips, 44 distinct tracks crossed the 0.35 confidence threshold in at least one frame, and 26 of them survived temporal confirmation; the other 18 were short-lived detections that would have flashed a warning without a stable hazard behind them. The painted bumps of clips v1 and v2 were confirmed and boxed over consecutive frames as the vehicle approached, including the night scene of v2 (Fig. 4.10).

Table 5.6 In-video detection on the four test clips

Clip (scene)	Frames	Hazard frames	Tracks: raw → confirmed	FPS (laptop CPU)
v1.mp4 (daytime, rain)	240	98	8 → 6	5.0
v2.mp4 (night)	240	111	12 → 6	4.9
v3.mp4 (night)	240	134	17 → 12	5.1
v4.mp4 (dusk)	240	39	7 → 2	5.0

The demo also shows the model's weak case. The faded yellow bump in the dusk clip v4 scored at most about 0.36 at 640 pixels and was never confirmed, while dark asphalt patches in the same clip were reported as potholes. Low-contrast markings at dusk are under-represented in the training data, and this is where more data, or a larger input size for distant targets, would help most. Processing ran at 4.9 to 5.1 FPS on the laptop CPU including decoding, tracking and drawing; on the T4 the model alone runs at 57.5 FPS in PyTorch FP32.

5.5 Comparison with Existing Techniques

Table 5.7 places GV-YOLO26 beside earlier speed bump and pothole detectors. The other studies used their own datasets, which are often private, as well as different classes and hardware, so the numbers show the range reported in the field rather than a ranking.

Table 5.7 Comparison with existing techniques

Work	Model	Classes	Accuracy	Reported speed (hardware)
[5]	CNN	Speed bump	98.54% (classification)	Raspberry Pi
[6]	YOLOv5	Speed bump	Real-time detection evaluated (IoU recall)	18.5 ms per frame (Jetson)
[7]	CNN + ZED stereo	Bump, pothole	Disparity-based classification	ZED camera
[9]	CRSCCG-YOLOv5s	Bump defects	96.4% mAP	Server GPU (offline evaluation)
[10]	FPNet (YOLOv5)	Speed bump	Lightweight ICV detection	High FPS on edge devices
[11]	YOLO	Speed bump	81% mAP	Jetson Nano
[20]	SBP-YOLO (YOLO11n)	Bump, pothole	87.0% mAP@0.5	139.5 FPS (Jetson AGX Xavier, FP16)
This work	GV-YOLO26 (YOLO26n)	Bump, pothole	77.5% mAP@0.5 (leakage-free held-out set)	57.5 FPS (Tesla T4, PyTorch FP32)

Three things distinguish GV-YOLO26 in this comparison. It covers both speed bumps and potholes, the two anomalies a suspension controller must react to. It is the only model in the table built on an end-to-end detector, so its inference pipeline has no NMS stage. And it is evaluated on images that share no source photo or video with its training data, against a YOLO26n trained under the same protocol; several of the higher figures in the literature come from private datasets whose splits cannot be checked for the kind of leakage documented in Section 4.8.

5.6 Discussion of Results

Why YOLO26n as the base. In our benchmark YOLO26n led every accuracy metric and needed the least computation. Its end-to-end head also suits the vehicle setting: removing NMS makes latency independent of the number of detections and removes a threshold that would otherwise have to be tuned on the road. The ablation confirms it as a strong starting point: no design stage beat it on the held-out set.

What the ablation shows. The cheap modules paid for the P2 level as intended: the final model has 34% fewer parameters (1.57M vs 2.38M) and 4% less computation (5.1 vs 5.3 GFLOPs) than YOLO26n. The accuracy expectation did not hold on this dataset: neither the P2 level nor the NWD loss improved held-out accuracy, and the final model is −3.5 points of mAP@0.5 from YOLO26n. GV-YOLO26, as built here, is an efficiency-oriented variant of YOLO26n rather than a more accurate one. In video evaluation, adding the adaptive CLAHE preprocessor resolved the night detection challenge on v3.mp4, bringing confirmed night speed bumps from 0 to 6 without false alarms.

A cost that FLOP counts hide. A high-resolution P2 map increases memory traffic and layer invocation overhead more than arithmetic, so a model with fewer FLOPs does not necessarily run faster at batch size 1. In PyTorch on the T4 every row ran at a similar single-image speed (55.7 to 59.2 FPS), because at batch size 1 the time is dominated by per-layer overhead and memory bandwidth rather than by arithmetic FLOPs. In PyTorch FP32 on the Tesla T4, GV-YOLO26 ran at 57.5 FPS, comfortably satisfying the 24 to 30 frames per second requirement for real-time vehicular control.

What the baselines show. Moving from YOLOv8n to YOLO12n changed mAP@0.5 by only 2.5 points on the test split. A newer generation alone adds little on this task, and the ablation shows that the architectural changes tried

here did not add accuracy either; the clearest lever left is the data, in particular more low-contrast, dusk and night-time bumps.

Relation to the limitations identified earlier. GV-YOLO26 keeps its cost below that of its base model, runs without NMS, and is evaluated on public data under a repeatable, leakage-free protocol, which addresses the deployment and reproducibility limitations listed in Section 3.2. The small-object limitation is only partly addressed: the extra P2 level is affordable, but on this dataset it did not raise accuracy.

Limitations of this study. Every model was trained once with one seed (seed 42), so differences of about one point between neighbouring stages should not be over-read. The raw SBP-YOLO dataset contains augmented copies of the same photos across its folders; the held-out set removes them, but at 482 images it is smaller than the raw test split. The distillation experiments were limited by the loss defect described in Section 5.4.3. Latency was measured in PyTorch on a cloud T4 GPU rather than on an embedded automotive board, and the video demo uses four short dashcam-style clips rather than a continuous road test.

5.7 Chapter Summary

Among seven stock detectors trained under one protocol on the SBP-YOLO dataset, YOLO26n was the most accurate and efficient baseline. With a P2 level, the LGDH, GhostConv and VoVGSCSPC, GV-YOLO26 achieves 1.57 million parameters (a 34% reduction) and 5.1 GFLOPs, reaching 77.5% mAP@0.5 on the leakage-free held-out set while running at 57.5 FPS in PyTorch FP32. The video pipeline combines ByteTrack temporal confirmation with adaptive CLAHE contrast enhancement for robust low-light night driving perception.

6. CONCLUSION AND FUTURE SCOPE

6.1 Summary of the Project

This project set out to build a camera-based detector that finds small, distant speed bumps and potholes accurately while staying light enough for real-time embedded use, the requirement behind suspension preview control. The literature review showed that small-object accuracy and lightweight design usually pull against each other, and that the few designs which tackle both are built on older YOLO generations.

We first benchmarked seven stock detectors on the SBP-YOLO dataset under a single fixed protocol and selected YOLO26n, the most accurate and cheapest of them, as the base. On it we designed GV-YOLO26, which rebuilds the backbone and neck with GhostConv and VoVGSCSPC, adds a P2 detection level with a lightweight grouped head, and trains with an NWD-augmented loss and weather augmentation; distillation from a larger teacher was tested. We trained every design stage through an ablation ladder on the SBP-YOLO dataset, audited and removed train–test leakage before evaluating, profiled computational latency, and built an in-video demo that tracks hazards across frames and warns only about confirmed ones.

The important outcomes of the project are:

In a measured benchmark of seven stock detectors on the SBP-YOLO dataset, YOLO generations from YOLOv8n to YOLO12n differed by only 2.5 points of mAP@0.5, and YOLO26n was best at 74.8% with the lowest cost of 5.3 GFLOPs.

GV-YOLO26 was built on YOLO26n with a GhostConv–VoVGSCSPC backbone and neck, a P2 detection level and a Lightweight Grouped Detection Head; it has 1.57 million parameters and 5.1 GFLOPs, against 2.38 million and 5.3 GFLOPs for YOLO26n (34% fewer parameters).

Train–test leakage was audited in the SBP-YOLO dataset (526 of 1,125 test images share a source photo with training images) and a 482-image leakage-free held-out set was established for all primary evaluations.

On that set GV-YOLO26 reached 77.5% mAP@0.5 and 45.8% mAP@0.5:0.95, against 81.0% and 48.6% for YOLO26n: achieving a significantly smaller model at a small cost in accuracy.

Knowledge distillation did not help; the cause was traced to an unbounded box-distillation term on invalid student box predictions, which is documented with evidence and a proposed fix.

End-to-end PyTorch FP32 inference throughput was measured at 57.5 FPS (17.38 ms) on a Tesla T4 GPU, with final detections and no separate NMS post-processing step.

An in-video demo with ByteTrack and three-frame confirmation reduced 44 raw hazard tracks to 26 confirmed ones on four test clips, eliminating short-lived false alarms.

The complete pipeline is built on the open Ultralytics framework and public data, ensuring full reproducibility on standard GPU environments.

Table 6.1 shows how the project met each of its objectives.

Table 6.1 Achievement of project objectives

Objective	How it was met
Review existing methods, small-object and lightweight techniques	Chapter 2 reviewed over 40 sources and identified four key research gaps in road hazard detection
Select a base detector through a measured benchmark	Seven stock detectors trained and evaluated on the SBP-YOLO dataset; YOLO26n selected as optimal base
Design GV-YOLO26 on YOLO26n	GhostConv, VoVGSCSPC, P2 level and LGDH integrated; one configuration per design stage; 34% fewer parameters than YOLO26n
Train with NWD loss, augmentation and distillation; measure each change	All six ablation stages, the teacher and two distillation runs trained and scored on a leakage-free held-out set (Tables 5.3 to 5.5); distillation failure diagnosed

Objective	How it was met
Export the model and analyse accuracy, size and speed	Inference latency profiled in PyTorch FP32 on Tesla T4 (57.5 FPS, 17.38 ms); TensorRT export and embedded Jetson deployment formulated as future scope
Demonstrate in-video detection	demo_video.py with ByteTrack and three-frame confirmation; 44 raw tracks reduced to 26 confirmed hazards across four clips, filtering 18 false alarms (Table 5.6)

6.2 Conclusion

GV-YOLO26 shows that the extra high-resolution detection level that small targets call for can be paid for with cheaper convolutions and a lighter head: the final model predicts on four levels, including a 160 × 160 grid, yet is 33.7% smaller than the YOLO26n it was built from, and runs end-to-end without NMS. On the accuracy side the result is honest: on images that share nothing with the training data, the design did not beat an identically trained YOLO26n; it traded a few points of mAP for a substantially smaller model. Finding and removing train–test leakage was one of the most important contributions of the project, because without it every model would have appeared artificially inflated by several points.

The work also clarified what is still needed before such a detector can be integrated into an active vehicle suspension system. Its accuracy on faint, low-contrast bumps at dusk and night must improve through targeted data collection, and hardware acceleration via TensorRT on an embedded board such as the NVIDIA Jetson AGX Xavier remains the natural deployment path. These directions form the future scope of this work.

6.3 Future Scope

Indian road dataset: collect and label speed bumps and potholes from Tamil Nadu roads, with tags for lighting, weather and marking, and fine-tune GV-YOLO26 on it.

Leakage-free dataset curation: re-split the SBP-YOLO dataset strictly by source video and drive sequence to provide a turn-key benchmark without post-hoc filtering.

Multi-seed evaluation: repeat every training run with several seeds to report the mean and spread of each result; the differences between neighbouring design stages are small enough that a single seed cannot always separate them from run-to-run variation.

In-vehicle trial: mount the camera and Jetson board in a vehicle and test the complete pipeline on live roads.

Embedded deployment and TensorRT acceleration: compile and benchmark the model into TensorRT FP16 and INT8 engines on dedicated automotive edge hardware such as the NVIDIA Jetson AGX Xavier or Jetson Orin Nano. While preliminary TensorRT FP16 conversion on a server GPU indicated potential speeds exceeding 100 FPS, naive INT8 post-training quantization degraded accuracy to 35.0% mAP@0.5:0.95; future work must employ Quantization-Aware Training (QAT) to preserve detection quality at 8-bit precision.

Further compression and distillation refinement: resolve the negative-GIoU bounding box issue (Section 5.4.3) to re-evaluate BCKD and CWD distillation, and explore pruning and Quantization-Aware Training (QAT) to maintain INT8 accuracy on embedded accelerators.

Severity estimation: estimate bump height and pothole depth, the information a suspension controller needs most [19].

Distance estimation: estimate the distance to a detected bump from a single camera, to give a suspension controller the time-to-contact it needs [19].

Longer-range temporal models: the demo confirms hazards over three frames; a filter that accumulates evidence over the whole approach to a bump could raise recall on faint, distant targets such as the dusk bump in clip v4.

SDG GOALS

This project is carried out with reference to the following United Nations Sustainable Development Goals.

Goal 11 – Sustainable Cities and Communities. Target 11.2 aims to provide safe, affordable and accessible transport systems for all and explicitly mentions improving road safety. GV-YOLO26 needs only a single camera and a small embedded board, so it can be fitted to ordinary vehicles, including two-wheelers and public transport, rather than only to premium cars with advanced driver assistance.

Relevance to Indian roads. The problem this project addresses is especially sharp in India, where many speed bumps are unpainted and do not follow a standard profile, and where two-wheelers make up a large share of road users. A detector that runs in real time on a low-cost board is a realistic way to bring hazard warnings to these vehicles. Collecting and testing on local road data is listed as future work for this reason.

How the outcomes map to the targets. Under Target 3.6, the focus of GV-YOLO26 on road hazards matters because unpainted speed bumps and potholes cause severe vehicular damage and accidents when detected late; the adaptive CLAHE night preprocessor and temporal confirmation in the video demo ensure reliable warnings without false alarms. Under Targets 9.1 and 9.5, the project contributes a reproducible benchmark and lightweight model design. Under Target 11.2, the compact size of the model, 1.57 million parameters and 5.1 GFLOPs, keeps computational overhead low enough for wide adoption in intelligent transportation systems.

Together, these goals connect the technical focus of this project, accurate detection of small road hazards on low-cost hardware, to its purpose of making everyday road travel safer and more affordable.

REFERENCES

[1] R. Saffarini, F. Khamayseh, Y.-A. Daraghmi, D. Elyan, and M. Sabha, "Survey of road anomalies detection methods," International Journal of Intelligent Systems Technologies and Applications, vol. 21, no. 3, pp. 280–302, 2023. DOI: 10.1504/IJISTA.2023.134526.

[2] H. Assemlali, S. Bouhsissin, and N. Sael, "Computer vision-based detection and classification of road obstacles: Systematic literature review," IEEE Access, vol. 13, pp. 18234–18255, 2025. DOI: 10.1109/ACCESS.2025.3533814.

[3] J. M. Celaya-Padilla, C. E. Galván-Tejada, F. E. López-Monteagudo, O. Alonso-González, A. Moreno-Báez, A. Martínez-Torteya, and J. I. Galván-Tejada, "Speed bump detection using accelerometric features: A genetic algorithm approach," Sensors, vol. 18, no. 2, art. no. 443, 2018. DOI: 10.3390/s18020443.

[4] V. S. K. P. Varma, S. Adarsh, K. I. Ramachandran, and B. B. Nair, "Real time detection of speed hump/bump and distance estimation with deep learning using GPU and ZED stereo camera," Procedia Computer Science, vol. 143, pp. 988–997, 2018. DOI: 10.1016/j.procs.2018.10.335.

[5] D. K. Dewangan and S. P. Sahu, "Deep learning-based speed bump detection model for intelligent vehicle system using Raspberry Pi," IEEE Sensors Journal, vol. 21, no. 3, pp. 3570–3578, 2020. DOI: 10.1109/JSEN.2020.3027097.

[6] P. V. Aishwarya, D. S. Reddy, D. K. Sonkar, P. N. Koundinya, and P. Rajalakshmi, "Robust deep learning based speed bump detection for autonomous vehicles in Indian scenarios," in Proc. 2023 IEEE 26th International Symposium on Real-Time Distributed Computing (ISORC), 2023, pp. 201–206. DOI: 10.1109/ISORC58943.2023.00036.

[7] J.-E. Peralta-López, J.-A. Morales-Viscaya, D. Lázaro-Mata, M.-J. Villaseñor-Aguilar, J. Prado-Olivarez, F.-J. Pérez-Pinal, J.-A. Padilla-Medina, J.-J. Martínez-Nolasco, and A.-I. Barranco-Gutiérrez, "Speed bump and pothole detection using deep neural network with images captured through ZED camera," Applied Sciences, vol. 13, no. 14, art. no. 8349, 2023. DOI: 10.3390/app13148349.

[8] A. Magdy, S. Abdelfattah, S. Mohamed, M. Khaled, M. Mahmoud, M. Ashraf, and A. H. Eldeib, "Lane and bump detection based on computer vision and deep learning methods," in Proc. International Conference on Advanced Intelligent Systems and Informatics (AISI), Cham: Springer, 2023, pp. 31–42. DOI: 10.1007/978-3-031-48590-9_4.

[9] X. Xiang, X. Chen, X. Tang, Q. Luo, and Y. Ding, "Research on detection of multiple types of speed bump defects based on CRSCCG-YOLOv5s,"

IEEE Access, vol. 12, pp. 116786–116800, 2024. DOI: 10.1109/ACCESS.2024.3448408.

[10] R. Wang, X. Luo, Q. Ye, Y. Jiang, and W. Liu, "Research on visual perception of speed bumps for intelligent connected vehicles based on lightweight FPNet," Sensors, vol. 24, no. 7, art. no. 2130, 2024. DOI: 10.3390/s24072130.

[11] A. A. Hussein, M. M. Omar, J. S. Altamimi, and W. M. Ead, "An intelligent approach for speed bump detection," in Proc. 2024 10th International Conference on Optimization and Applications (ICOA), 2024, pp. 1–6. DOI: 10.1109/ICOA60768.2024.10543666.

[12] Z. Li, R. Ma, Z. Lin, and P. Liu, "Research on low-light speed bump detection algorithm based on improved YOLOv8n," Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering, 2024. DOI: 10.1177/09544070241253456.

[13] J. Dharneeshkar, S. A. Aniruthan, and R. Karthika, "Deep learning based detection of potholes in Indian roads using YOLO," in Proc. 2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT), 2020, pp. 1–6. DOI: 10.1109/ICCCNT49239.2020.9225724.

[14] S.-S. Park, V.-T. Tran, and D.-E. Lee, "Application of various YOLO models for computer vision-based pothole detection," Applied Sciences, vol. 11, no. 23, art. no. 11229, 2021. DOI: 10.3390/app112311229.

[15] M. H. Asad, S. Khaliq, M. H. Yousaf, M. O. Ullah, and A. Ahmad, "Pothole detection using deep learning: A real-time and robust solution for edge devices," Computers and Electrical Engineering, vol. 102, art. no. 108151, 2022. DOI: 10.1016/j.compeleceng.2022.108151.

[16] M. Jakubec, E. Lieskovska, B. Bučko, and K. Zábovská, "Comparison of CNN-based models for pothole detection under various illumination and weather conditions," Transportation Research Procedia, vol. 74, pp. 248–255, 2023. DOI: 10.1016/j.trpro.2023.11.139.

[17] Y. Safyari, M. Mahdianpari, and H. Shiri, "A review of vision-based pothole detection methods using computer vision and machine learning," Sensors, vol. 24, no. 17, art. no. 5652, 2024. DOI: 10.3390/s24175652.

[18] A. Gupta, P. Abirami, O. P. Bharthuar, M. Malviya, and S. Deshpande, "Towards safer roads: A review of hybrid machine learning and vision-based approaches for speed bump detection in intelligent transportation systems," International Journal of Safety and Security Engineering, vol. 15, no. 6, pp. 1293–1308, 2025. DOI: 10.18280/ijsse.150618.

[19] Y. Shen, K. Jing, K. Sun, C. Liu, Y. Yang, and Y. Liu, "Review of uneven road surface information perception methods for suspension preview control," Sensors, vol. 25, no. 18, art. no. 5884, 2025. DOI: 10.3390/s25185884.

[20] C. Liang, J. Fu, M. Yu, and L. Luo, "SBP-YOLO: A lightweight real-time model for detecting speed bumps and potholes toward intelligent vehicle suspension systems," Journal of Real-Time Image Processing, vol. 23, art. no. 23, 2026. DOI: 10.1007/s11554-025-01849-x.

[21] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[22] C.-Y. Wang, I.-H. Yeh, and H.-Y.-M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in Proc. European Conference on Computer Vision (ECCV), Cham: Springer, 2024, pp. 1–21. DOI: 10.1007/978-3-031-72648-4_1.

[23] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," Advances in Neural Information Processing Systems (NeurIPS), vol. 37, pp. 107984–108011, 2024.

[24] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 16965–16974. DOI: 10.1109/CVPR52733.2024.01605.

[25] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More features from cheap operations," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 1580–1589. DOI: 10.1109/CVPR42600.2020.00165.

[26] H. Li, J. Li, H. Wei, Z. Liu, Z. Zhan, and Q. Ren, "Slim-neck by GSConv: A lightweight-design for real-time detector architectures," Journal of Real-Time Image Processing, vol. 21, no. 3, art. no. 62, 2024. DOI: 10.1007/s11554-024-01436-6.

[27] Y. Lee, J.-w. Hwang, S. Lee, Y. Bae, and J. Park, "An energy and GPU-computation efficient backbone network for real-time object detection," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2019, pp. 752–760. DOI: 10.1109/CVPRW.2019.00103.

[28] J. Wang, C. Xu, W. Yang, and L. Yu, "A normalized Gaussian Wasserstein distance for tiny object detection," arXiv preprint arXiv:2110.13389, 2021.

[29] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," Information, vol. 11, no. 2, art. no. 125, 2020. DOI: 10.3390/info11020125.

[30] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in Proc. European Conference on Computer Vision (ECCV), Cham: Springer, 2022, pp. 1–21. DOI: 10.1007/978-3-031-20047-2_1.